

# Documentación del dominio de contratos

Jasrapo Backend

20 de septiembre de 2026

## Índice

|                                                                         |           |
|-------------------------------------------------------------------------|-----------|
| <b>Flujo transversal</b>                                                | <b>5</b>  |
| <b>Introducción</b>                                                     | <b>5</b>  |
| <b>1. Alcance y convenciones</b>                                        | <b>5</b>  |
| 1.1. Decisión funcional propuesta sobre novedades . . . . .             | 5         |
| 1.2. Convenciones técnicas . . . . .                                    | 6         |
| <b>2. Clientes</b>                                                      | <b>6</b>  |
| 2.1. Alcance y entradas HTTP . . . . .                                  | 6         |
| 2.2. Comportamiento de negocio verificable . . . . .                    | 6         |
| 2.3. Ejemplo JSON . . . . .                                             | 7         |
| 2.4. Estados . . . . .                                                  | 7         |
| 2.4.1. Validación de identificación por algoritmo verificador . . . . . | 8         |
| 2.4.2. Singleton CONSUMIDOR FINAL . . . . .                             | 8         |
| 2.5. Efectos y transacciones . . . . .                                  | 8         |
| 2.6. Gráfico de estados . . . . .                                       | 8         |
| 2.7. Casos de uso . . . . .                                             | 9         |
| 2.7.1. Caso: Catálogo de tipos de identificación . . . . .              | 9         |
| 2.7.2. Caso: Crear cliente . . . . .                                    | 9         |
| 2.7.3. Caso: Listar clientes . . . . .                                  | 10        |
| 2.7.4. Caso: Obtener cliente . . . . .                                  | 10        |
| 2.7.5. Caso: Actualizar cliente . . . . .                               | 11        |
| 2.7.6. Caso: Eliminar cliente . . . . .                                 | 11        |
| 2.8. Pendientes funcionales . . . . .                                   | 12        |
| 2.9. No documentado o pendiente de confirmar . . . . .                  | 12        |
| <b>3. Contratos</b>                                                     | <b>12</b> |
| 3.1. Alcance y entradas HTTP . . . . .                                  | 12        |
| 3.2. Ejemplo JSON . . . . .                                             | 13        |
| 3.3. Estados . . . . .                                                  | 13        |
| 3.4. Efectos y transacciones . . . . .                                  | 13        |
| 3.4.1. Cadena POST y efectos . . . . .                                  | 14        |
| 3.4.2. Fórmula de instalación . . . . .                                 | 14        |
| 3.5. Gráfico de estados . . . . .                                       | 14        |
| 3.5.1. Estado de cobranza . . . . .                                     | 15        |
| 3.6. Casos de uso . . . . .                                             | 15        |
| 3.6.1. Caso: Crear contrato . . . . .                                   | 15        |
| 3.6.2. Caso: Listar contratos . . . . .                                 | 15        |
| 3.6.3. Caso: Obtener contrato . . . . .                                 | 16        |

|           |                                                            |           |
|-----------|------------------------------------------------------------|-----------|
| 3.6.4.    | Caso: Actualizar contrato . . . . .                        | 16        |
| 3.6.5.    | Caso: Finalizar vínculo de medidor . . . . .               | 16        |
| 3.6.6.    | Caso: Asignar ruta de instalación . . . . .                | 17        |
| 3.6.7.    | Caso: Eliminar contrato . . . . .                          | 17        |
| 3.6.8.    | Caso: Generar PDF de solicitud de conexión . . . . .       | 17        |
| 3.6.9.    | Caso: Generar PDF de acuerdo de responsabilidad . . . . .  | 17        |
| 3.7.      | No documentado o pendiente de confirmar . . . . .          | 18        |
| <b>4.</b> | <b>Convenios de pago</b>                                   | <b>18</b> |
| 4.1.      | Alcance y entradas HTTP . . . . .                          | 18        |
| 4.2.      | Ejemplo JSON . . . . .                                     | 19        |
| 4.3.      | Estados . . . . .                                          | 19        |
| 4.4.      | Efectos y transacciones . . . . .                          | 20        |
| 4.4.1.    | Tablas y fórmulas confirmadas . . . . .                    | 20        |
| 4.5.      | Transiciones de estado del convenio . . . . .              | 20        |
| 4.5.1.    | Cadena de transiciones confirmada . . . . .                | 20        |
| 4.5.2.    | Handlers de efectos derivados . . . . .                    | 21        |
| 4.5.3.    | Restricciones operativas . . . . .                         | 21        |
| 4.6.      | Gráfico de estados . . . . .                               | 21        |
| 4.6.1.    | Estado de la cuota . . . . .                               | 22        |
| 4.7.      | Casos de uso . . . . .                                     | 22        |
| 4.7.1.    | Caso: Consultar estados de convenio . . . . .              | 22        |
| 4.7.2.    | Caso: Consultar estados de cuota . . . . .                 | 22        |
| 4.7.3.    | Caso: Consultar resumen de deuda . . . . .                 | 22        |
| 4.7.4.    | Caso: Crear convenio . . . . .                             | 23        |
| 4.7.5.    | Caso: Listar convenios . . . . .                           | 23        |
| 4.7.6.    | Caso: Obtener convenio . . . . .                           | 23        |
| 4.7.7.    | Caso: Listar cuotas de un convenio . . . . .               | 23        |
| 4.7.8.    | Caso: Actualizar estado del convenio . . . . .             | 24        |
| 4.7.9.    | Caso: Anular convenio . . . . .                            | 24        |
| 4.7.10.   | Caso: Generar PDF del convenio . . . . .                   | 24        |
| 4.8.      | No documentado o pendiente de confirmar . . . . .          | 24        |
| <b>5.</b> | <b>Rutas y órdenes de trabajo</b>                          | <b>25</b> |
| 5.1.      | Alcance y entradas HTTP . . . . .                          | 25        |
| 5.2.      | Comportamiento de negocio verificable . . . . .            | 26        |
| 5.3.      | Ejemplo JSON . . . . .                                     | 26        |
| 5.4.      | Estados . . . . .                                          | 27        |
| 5.4.1.    | Máquina de estados de la Ruta . . . . .                    | 27        |
| 5.5.      | Regla de novedades sin estado adicional de orden . . . . . | 27        |
| 5.5.1.    | Flujo acordado de orden con novedad . . . . .              | 27        |
| 5.5.2.    | Tres máquinas separadas . . . . .                          | 28        |
| 5.5.3.    | Ejemplo de respuesta derivada . . . . .                    | 29        |
| 5.5.4.    | Regla de backend y pendiente funcional . . . . .           | 29        |
| 5.6.      | Efectos y transacciones . . . . .                          | 29        |
| 5.7.      | Flujos acordados y máquinas de estado . . . . .            | 30        |
| 5.7.1.    | Estado de la orden . . . . .                               | 30        |
| 5.7.2.    | Estado de las novedades de órdenes de trabajo . . . . .    | 31        |
| 5.8.      | Casos de uso . . . . .                                     | 31        |
| 5.8.1.    | Caso: Consultar períodos . . . . .                         | 31        |
| 5.8.2.    | Caso: Consultar lecturas elegibles . . . . .               | 32        |
| 5.8.3.    | Caso: Consultar lecturas de ruta . . . . .                 | 32        |

|           |                                                          |           |
|-----------|----------------------------------------------------------|-----------|
| 5.8.4.    | Caso: Crear ruta . . . . .                               | 32        |
| 5.8.5.    | Caso: Listar rutas . . . . .                             | 33        |
| 5.8.6.    | Caso: Obtener ruta . . . . .                             | 33        |
| 5.8.7.    | Caso: Actualizar ruta . . . . .                          | 33        |
| 5.8.8.    | Caso: Reasignar ruta . . . . .                           | 33        |
| 5.8.9.    | Caso: Eliminar ruta . . . . .                            | 34        |
| 5.8.10.   | Caso: Exportar hoja de campo . . . . .                   | 34        |
| 5.8.11.   | Caso: Consultar órdenes de una ruta . . . . .            | 34        |
| 5.8.12.   | Caso: Actualizar estado de orden . . . . .               | 34        |
| 5.8.13.   | Caso: Vincular lectura a orden . . . . .                 | 35        |
| 5.8.14.   | Caso: Actualizar operación del operador . . . . .        | 35        |
| 5.8.15.   | Caso: Listar rutas del operador . . . . .                | 35        |
| 5.8.16.   | Caso: Actualizar estado de ruta del operador . . . . .   | 35        |
| 5.9.      | Pendientes funcionales . . . . .                         | 36        |
| 5.10.     | No documentado o pendiente de confirmar . . . . .        | 36        |
| <b>6.</b> | <b>Lecturas y consumo</b>                                | <b>36</b> |
| 6.1.      | Separación entre orden y lectura . . . . .               | 36        |
| 6.2.      | Alcance y entradas HTTP . . . . .                        | 37        |
| 6.3.      | Comportamiento de negocio verificable . . . . .          | 37        |
| 6.4.      | Ejemplo JSON de respuesta . . . . .                      | 37        |
| 6.5.      | Estados . . . . .                                        | 38        |
| 6.5.1.    | Máquina de estados de la Lectura . . . . .               | 39        |
| 6.6.      | Responsabilidades por estado . . . . .                   | 39        |
| 6.7.      | Efectos y transacciones . . . . .                        | 39        |
| 6.8.      | Cálculo de consumo . . . . .                             | 39        |
| 6.9.      | Estimación autorizada por secretaría . . . . .           | 40        |
| 6.10.     | Diagrama de flujo acordado . . . . .                     | 40        |
| 6.11.     | Casos de uso . . . . .                                   | 40        |
| 6.11.1.   | Caso: Listar lecturas . . . . .                          | 40        |
| 6.11.2.   | Caso: Consultar catálogo de estados . . . . .            | 41        |
| 6.11.3.   | Caso: Obtener lectura . . . . .                          | 41        |
| 6.11.4.   | Caso: Actualizar lectura administrativa . . . . .        | 42        |
| 6.11.5.   | Caso: Eliminar lectura . . . . .                         | 42        |
| 6.11.6.   | Caso: Listar lecturas del operador . . . . .             | 42        |
| 6.11.7.   | Caso: Listar anomalías pendientes del operador . . . . . | 42        |
| 6.11.8.   | Caso: Actualizar lectura del operador . . . . .          | 43        |
| 6.12.     | Pendientes funcionales . . . . .                         | 43        |
| 6.13.     | No documentado o pendiente de confirmar . . . . .        | 44        |
| <b>7.</b> | <b>Medidores</b>                                         | <b>44</b> |
| 7.1.      | Alcance y entradas HTTP . . . . .                        | 44        |
| 7.2.      | Comportamiento de negocio verificable . . . . .          | 44        |
| 7.3.      | Ejemplo JSON . . . . .                                   | 45        |
| 7.4.      | Estados . . . . .                                        | 45        |
| 7.4.1.    | Máquina de estados del Medidor . . . . .                 | 45        |
| 7.4.2.    | Estados internos de ReemplazoMedidor . . . . .           | 46        |
| 7.5.      | Efectos y transacciones . . . . .                        | 46        |
| 7.6.      | Gráfico de estados . . . . .                             | 47        |
| 7.7.      | Casos de uso . . . . .                                   | 47        |
| 7.7.1.    | Caso: Crear medidor . . . . .                            | 47        |
| 7.7.2.    | Caso: Listar medidores . . . . .                         | 47        |

|                                                        |           |
|--------------------------------------------------------|-----------|
| 7.7.3. Caso: Obtener medidor . . . . .                 | 47        |
| 7.7.4. Caso: Actualizar medidor . . . . .              | 48        |
| 7.7.5. Caso: Eliminar medidor . . . . .                | 48        |
| 7.7.6. Caso: Reemplazar medidor . . . . .              | 48        |
| 7.7.7. Caso: Obtener reemplazo de medidor . . . . .    | 48        |
| 7.7.8. Caso: Aprobar reemplazo de medidor . . . . .    | 49        |
| 7.7.9. Caso: Exportar inventario CSV . . . . .         | 49        |
| 7.7.10. Caso: Exportar inventario PDF . . . . .        | 49        |
| 7.7.11. Caso: Reportar defecto de medidor . . . . .    | 49        |
| 7.7.12. Caso: Dar de baja un medidor . . . . .         | 50        |
| 7.8. Pendientes funcionales . . . . .                  | 50        |
| 7.9. No documentado o pendiente de confirmar . . . . . | 50        |
| <b>8. Categorías de tarifa</b>                         | <b>50</b> |
| 8.1. Alcance y entradas HTTP . . . . .                 | 51        |
| 8.2. Comportamiento de negocio verificable . . . . .   | 51        |
| 8.3. Ejemplo JSON . . . . .                            | 51        |
| 8.4. Estados . . . . .                                 | 51        |
| 8.5. Efectos y transacciones . . . . .                 | 52        |
| 8.6. Estados y transiciones no encontrados . . . . .   | 52        |
| 8.7. Casos de uso . . . . .                            | 52        |
| 8.7.1. Caso: Crear categoría de tarifa . . . . .       | 52        |
| 8.7.2. Caso: Listar categorías . . . . .               | 52        |
| 8.7.3. Caso: Obtener categoría . . . . .               | 53        |
| 8.7.4. Caso: Actualizar categoría . . . . .            | 53        |
| 8.7.5. Caso: Eliminar categoría . . . . .              | 53        |
| 8.8. Pendientes funcionales . . . . .                  | 53        |
| 8.9. No documentado o pendiente de confirmar . . . . . | 54        |

## Flujo transversal

La cadena confirmada es contrato, prefactura de instalación, pago validado, instalación, lecturas/consumo, prefacturación mensual y convenios/pagos. La creación puede ejecutar `SELECT generar_prefactura_instalacion(...)`; `PagoValidadoHandler` lleva la instalación a `PENDIENTE_INSTALACION`; no se encontró una transición automática a `ACTIVO`. La fórmula mensual completa no fue encontrada en la evidencia revisada.



Figura 1: Flujo general del dominio.

## Introducción

Este documento consolida la documentación técnica de los ocho subdominios que participan en el ciclo de contratos del backend. Describe sus superficies HTTP, casos de uso, estados, persistencia, efectos y aspectos aún pendientes de confirmar. Los diagramas Mermaid de la fuente Markdown se conservan como bloques de código monoespaciado para evitar dependencias de ejecución externas.

### 1. Alcance y convenciones

La documentación distingue tres niveles: **Confirmada** (visible en código, Prisma, migración o handler registrado), **Inferida** (deducción que no prueba una escritura o transición) y **No encontrada** (la revisión no halló fórmula, handler, rollback, transición o integración). Las fórmulas sólo se presentan como vigentes cuando incluyen fuente, símbolo y línea aproximada. Los efectos directos son escrituras del caso de uso/repositorio; los indirectos provienen de handlers, eventos o funciones SQL.

#### 1.1. Decisión funcional propuesta sobre novedades

No se agregará un estado persistido `NOVEDAD` a `OrdenTrabajo`. La orden permanecerá en `EN_PROGRESO` mientras exista una novedad y la UI mostrará una alerta derivada de la relación de novedades activas. `OrdenTrabajo.estado` representa ejecución operativa; `NovedadOrdenTrabajo.estado`, análisis de anomalía; y `Lectura.estado`, captura y validación del dato. `NOVEDAD` es un indicador derivado, no un valor de enum. La decisión está **pendiente de implementación**; el backend deberá impedir `COMPLETADA` si una novedad abierta afecta el resultado, y queda pendiente definir el criterio de bloqueo (`afectaOrden`, tipo, resolución o regla equivalente).

## 1.2. Convenciones técnicas

La API usa el prefijo `/api/v1`; los `BigInt` se serializan como strings; `deletedAt` representa eliminación lógica; y “transaccional” sólo se usa cuando la transacción fue confirmada. La fuente primaria es el código implementado y el esquema Prisma.

## 2. Clientes

Subdominio de identidad y datos del titular. La API usa `/api/v1` y autenticación Bearer.

**Ficha de dominio:** documenta el alta, consulta, actualización y baja lógica de clientes, además del catálogo de tipos de identificación. El alcance cubre las superficies HTTP confirmadas en `ClientController`.

### 2.1. Alcance y entradas HTTP

- **Controlador:** `ClientController` (`backend/src/operations/clients/interfaces/http/client.controller`)
- **Servicio:** `ClientService`.
- **Casos:** `CreateClientUseCase`, `FindOneClientUseCase`, `UpdateClientUseCase`, `RemoveClientUseCase`.
- `GET /identification-types` no tiene caso de uso dedicado.
- `GET /clients` tampoco tiene `FindAllClientUseCase`: el servicio construye filtros y pagina directamente.

### 2.2. Comportamiento de negocio verificable

Las tablas relacionadas son `Clientes` y `CatalogoTiposIdentificacion`; `Contratos` consume la identidad del titular. La transición del contrato al completar una instalación (`detokenizeINSTALACION`) o reconexión (`detokenizeRECONEXION`) **sí** existe — está implementada vía `detokenizeapplyContractLifecycleTr` (`backend/src/operations/routes/infrastructure/repositories/prisma-orden-trabajo.repository.ts`) y documentada en `02-contratos.md`, sección “Transiciones de estado del contrato”. El flujo del cliente es alta, actualización, baja lógica y consulta del catálogo; las validaciones de identificación (algoritmo verificador) son ejecutadas en `detokenizeCreateClientUseCase` y `detokenizeUpdateClientUseCase`.

| Método y ruta                                            | Caso/servicio ejecutado                                                                                                       | Resultado             |
|----------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|-----------------------|
| GET<br><code>/api/v1/clients/identification-types</code> | <code>ClientService.findAllIdentifications()</code>                                                                           | Catálogo activo.      |
| POST <code>/api/v1/clients</code>                        | <code>ClientService.create()</code><br><code>CreateClientUseCase.execute()</code>                                             | Cliente creado (201). |
| GET <code>/api/v1/clients</code>                         | <code>ClientService.findAll()</code><br><code>buildClientFilters()</code><br><code>ClientRepository.paginateClientes()</code> | Página de clientes.   |
| GET <code>/api/v1/clients/:id</code>                     | <code>ClientService.findOne()</code><br><code>FindOneClientUseCase.execute()</code>                                           | Cliente encontrado.   |
| PATCH <code>/api/v1/clients/:id</code>                   | <code>ClientService.update()</code><br><code>UpdateClientUseCase.execute()</code>                                             | Cliente actualizado.  |
| DELETE <code>/api/v1/clients/:id</code>                  | <code>ClientService.delete()</code><br><code>RemoveClientUseCase.execute()</code>                                             | Soft delete.          |

## 2.3. Ejemplo JSON

```
{
  "clienteId": "10",
  "identificacion": "1712345678",
  "nombres": "Juan",
  "apellidos": "Pérez",
  "razonSocial": null,
  "email": "juan@example.com",
  "telefono": "0991234567",
  "direccionDomicilio": "Av. Principal 123",
  "activo": true
}
```

| Campo                               | Tipo        | Regla/uso                                  |
|-------------------------------------|-------------|--------------------------------------------|
| clienteId                           | string      | BigInt serializado.                        |
| tipoIdentificacionId                | number      | Referencia al catálogo.                    |
| identificacion                      | string      | Identificador del titular.                 |
| nombres, apellidos                  | string/null | Nombre personal.                           |
| razonSocial                         | string/null | Nombre empresarial cuando aplica.          |
| email, telefono, direccionDomicilio | string/null | Datos de contacto y domicilio.             |
| activo                              | boolean     | Disponibilidad lógica expuesta por el DTO. |

## 2.4. Estados

No existe enum vigente de estado del cliente. La disponibilidad se expresa con los siguientes campos:

| Campo              | Tipo                        | Significado                                                                                                                                                                          |
|--------------------|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| activo             | boolean                     | Habilitación lógica expuesta por el DTO. Por defecto true.                                                                                                                           |
| deletedAt          | DateTime?                   | Soft delete; <b>interno</b> para el flujo de consulta, pero <b>SÍ se expone</b> en el DTO ClientResponseDto. Un cliente puede tener activo=true y deletedAt != null simultáneamente. |
| aplicaTerceraEdad  | boolean                     | Calculado en create/update según TerceraEdadUtil.aplica(fechaNacimiento). Por defecto false.                                                                                         |
| aplicaDiscapacidad | boolean                     | Flag configurable desde el DTO. Por defecto false.                                                                                                                                   |
| telefonoSecundario | string?                     | Teléfono alternativo. Presente en el modelo y en el DTO.                                                                                                                             |
| tipoIdentificacion | CatalogoTiposIdentificacion | Relación al catálogo; el enum TipoIdentificacion tiene 5 valores (CEDULA, RUC, PASAPORTE, CONSUMIDOR\\_FINAL, IDENTIFICACION\\_EXTRANJERA).                                          |

No hay reactivación pública: el flujo POST /clientes reactiva un registro existente solo cuando el tipoIdentificacionId=4 (CONSUMIDOR\\\_FINAL) o cuando la identificacion ya existía con deletedAt != null.

### 2.4.1. Validación de identificación por algoritmo verificador

CreateClientUseCase y UpdateClientUseCase invocan TipoIdentificacionUtil.validar(codigo, identificacion) antes de persistir. Las reglas por tipo son:

| codigo (SRI)                               | Tipo                        | Regla                                                                                 |
|--------------------------------------------|-----------------------------|---------------------------------------------------------------------------------------|
| 05                                         | CEDULA                      | 10 dígitos, algoritmo módulo 10.                                                      |
| 04 (con terminación 001)                   | RUC persona natural         | Cédula + 001.                                                                         |
| 04 (con tercer dígito 9, terminación 001)  | RUC privado                 | Algoritmo módulo 11 con coeficientes [4,3,2,7,6,5,4,3,2].                             |
| 04 (con tercer dígito 6, terminación 0001) | RUC público                 | Algoritmo módulo 11 con coeficientes [3,2,7,6,5,4,3,2].                               |
| 08                                         | PASAPORTE                   | Sin validación de checksum (solo presencia).                                          |
| 09                                         | IDENTIFICACION\\_EXTRANJERA | Sin validación de checksum.                                                           |
| 07                                         | CONSUMIDOR\\_FINAL          | No requiere identificación; usa lógica de singleton con reactivación (ver más abajo). |

**Nota cross-doc:** la nota en ../HISTORIAS-DE-USUARIO.md, escenario HU-01, marca esta validación como “pendiente de implementación”. La implementación SÍ existe en backend/src/shared/utils/tipo-identificacion.util.ts. Esa nota está desactualizada.

### 2.4.2. Singleton CONSUMIDOR FINAL

El tipo de identificación con id=4 (codigo SRI 07, CONSUMIDOR\\\_FINAL) tiene semántica especial: existe **a lo sumo un** registro activo de este tipo en toda la tabla.

- CreateClientUseCase detecta tipoIdentificacionId === 4 y llama a clientRepository.reactivar()
- Si existe al menos un registro activo o soft-deleted, se reutiliza el más antiguo y se reactivan los datos del request (email, teléfono, dirección).
- Si hay más de un registro, los extras se marcan con deletedAt = now para mantener el invariante de singleton.

No requiere identificacion ni nombres/apellidos.

## 2.5. Efectos y transacciones

Crear, actualizar y eliminar escriben Clientes mediante sus casos de uso; eliminar usa soft delete. Las consultas, buildClientFilters y ClientResponseDto.fromEntity son transformaciones/serialización puras. No se confirmó una transacción adicional ni handler de eventos propio.

| Tabla Prisma                | Uso                       |
|-----------------------------|---------------------------|
| Clientes                    | Titular y soft delete.    |
| CatalogoTiposIdentificacion | Tipos activos y relación. |

## 2.6. Gráfico de estados

```
stateDiagram-v2
    [*] --> Activo: crear
    Activo --> Eliminado: DELETE (soft delete)
```

## 2.7. Casos de uso

### 2.7.1. Caso: Catálogo de tipos de identificación

**Descripción:** devuelve los tipos activos para formularios.

**HTTP y ruta:** GET /api/v1/clients/identification-types.

**Permiso:** clientes:read.

**Cadena:** ClientController.findAllIdentificaciones() ClientService.findAllIdentificaciones()  
ClientRepository.findActiveTipoIdentificaciones() (sin caso dedicado).

**Errores relevantes:** 401 no autenticado; 403 sin permiso.

**Efectos:** ninguno; consulta y mapeo puros.

**Prisma:** CatalogoTiposIdentificacion.

**Entrada:** sin body, path ni query.

**Salida:**

```
[
  {
    "tipoIdentificacionId": 1,
    "codigo": "CEDULA",
    "descripcion": "Cédula"
  }
]
```

### 2.7.2. Caso: Crear cliente

**Descripción:** registra un nuevo titular.

**HTTP y ruta:** POST /api/v1/clients.

**Permiso:** clientes:create.

**Cadena:** ClientController.create() ClientService.create() CreateClientUseCase.execute()  
ClientRepository.

**Errores relevantes:**

- 400 datos inválidos (incluye validación del algoritmo verificador según el tipo).
- 401.
- 403.
- **Conflicto de identificación** (ya activa) — EntityAlreadyExistsException.
- **Identificación inválida** según el algoritmo — InvalidDomainOperationException.
- **Campos requeridos faltantes** según el tipo (nombres/apellidos/direccionDomicilio para tipos genéricos; razonSocial adicional para RUC).

**Efectos:**

- Alta en Clientes **o reactivación** si la identificación existía soft-deleted.
- Cálculo automático de aplicaTerceraEdad vía TerceraEdadUtil.aplica(fechaNacimiento).
- Normalización: emails a lowercase, nombres/apellidos/razonSocial a uppercase, identificación con trim.
- El DTO de salida es serialización pura.

**Prisma:** Clientes, CatalogoTiposIdentificacion.

**Entrada:**

```
{
  "tipoIdentificacionId": 1,
  "identificacion": "1712345678",
  "nombres": "Juan",
  "apellidos": "Pérez",
  "razonSocial": null,
}
```

```
"email": "juan@example.com"
}
```

**Salida:**

```
{
  "clienteId": "10",
  "identificacion": "1712345678",
  "nombres": "Juan",
  "apellidos": "Pérez",
  "razonSocial": null,
  "email": "juan@example.com",
  "telefono": null,
  "direccionDomicilio": null,
  "activo": true
}
```

### 2.7.3. Caso: Listar clientes

**Descripción:** consulta clientes con filtros opcionales y paginación.

**HTTP y ruta:** GET /api/v1/clients.

**Permiso:** clientes:read.

**Cadena:** ClientController.findAll() ClientService.findAll() buildClientFilters()  
ClientRepository.paginateClientes(); no existe FindAllClientUseCase.

**Errores relevantes:** 400 query inválida; 401; 403.

**Efectos:** ninguno; consulta y DTO puros.

**Prisma:** Clientes, CatalogoTiposIdentificacion.

**Entrada:** sin body. Query: page, limit, identificacion, nombres, apellidos, nombreCompleto.

**Salida:**

```
{
  "data": [
    {
      "clienteId": "10",
      "identificacion": "1712345678",
      "nombres": "Juan",
      "apellidos": "Pérez",
      "activo": true
    }
  ],
  "meta": {
    "page": 1,
    "limit": 10,
    "total": 1,
    "totalPages": 1
  }
}
```

### 2.7.4. Caso: Obtener cliente

**Descripción:** devuelve un cliente por identificador.

**HTTP y ruta:** GET /api/v1/clients/:id.

**Permiso:** clientes:read.

**Cadena:** ClientController.findOne() ClientService.findOne() FindOneClientUseCase.execute()  
ClientRepository.

**Errores relevantes:** 400 ID inválido; 401; 403; 404 no encontrado.

**Efectos:** ninguno; DTO puro.

**Prisma:** Clientes, CatalogoTiposIdentificacion.

**Entrada:** sin body. Path: { "id": "10" }.

**Salida:**

```
{
  "clienteId": "10",
  "identificacion": "1712345678",
  "nombres": "Juan",
  "apellidos": "Pérez",
  "razonSocial": null,
  "email": "juan@example.com",
  "telefono": null,
  "direccionDomicilio": null,
  "activo": true
}
```

### 2.7.5. Caso: Actualizar cliente

**Descripción:** modifica parcialmente los datos permitidos del titular.

**HTTP y ruta:** PATCH /api/v1/clients/:id.

**Permiso:** clientes:update.

**Cadena:** ClientController.update() ClientService.update() UpdateClientUseCase.execute()  
ClientRepository.

**Errores relevantes:** 400 ID/body inválidos; 401; 403; 404 no encontrado.

**Efectos:** actualización de Clientes; DTO puro.

**Prisma:** Clientes, CatalogoTiposIdentificacion.

**Entrada:** path { "id": "10" }; body:

```
{
  "telefono": "0990000000",
  "email": "nuevo@example.com"
}
```

**Salida:**

```
{
  "clienteId": "10",
  "telefono": "0990000000",
  "email": "nuevo@example.com",
  "activo": true
}
```

### 2.7.6. Caso: Eliminar cliente

**Descripción:** marca el cliente como eliminado sin borrado físico.

**HTTP y ruta:** DELETE /api/v1/clients/:id.

**Permiso:** clientes:delete.

**Cadena:** ClientController.delete() ClientService.delete() RemoveClientUseCase.execute()  
ClientRepository.

**Errores relevantes:** 400 ID inválido; 401; 403; 404 no encontrado.

**Efectos:** soft delete en Clientes; **solo** setea deletedAt = now() (no toca activo). Tras la operación, el cliente tiene activo=true y deletedAt != null simultáneamente.

**Prisma:** Clientes.

**Entrada:** sin body. Path: { "id": "10" }.

**Salida:** objeto ClientResponseDto con los 13 campos del DTO, donde deletedAt está poblado y activo sigue true.

## 2.8. Pendientes funcionales

Bloqueos confirmados como pendientes en el código revisado. Cuando se cierre cada uno, sacar de acá y mover a la sección correspondiente.

- **Forma exacta de CreateClientDto y UpdateClientDto** (campos opcionales vs obligatorios, validaciones anidadas como @IsEmail, @Matches). Confirmar contra el código actual; la lista de campos en este doc es funcional, no de validación.
- **Forma exacta del query string de GET /api/v1/clients** (page, limit, identificacion, nombres, apellidos, nombreCompleto). Confirmar que el filtro nombreCompleto sigue vigente en el repositorio (vista en client-filters.mapper.ts).
- **Reactivación pública de clientes soft-deleted.** Existe **solo** para CONSUMIDOR\\\_FINAL y como efecto secundario al crear con la misma identificación. No hay endpoint público para reactivar un cliente regular; si se requiere, hay que decidir el criterio (?solo admin?) y agregar el endpoint.

## 2.9. No documentado o pendiente de confirmar

- Handlers de eventos de clientes. No hay event dispatcher que reaccione a cambios en Clientes. Si se requiere propagar a Contratos automáticamente (por ejemplo, cambio de dirección), hay que implementar el patrón outbox como en pagos.
- Estados legacy distintos de activo y deletedAt. Confirmar contra seeds y migraciones si existe alguna columna legacy.

## 3. Contratos

Vínculo entre cliente, medidor, tarifa y servicio.

**Ficha de dominio:** documenta el ciclo contractual, el vínculo con medidores, la asignación de rutas de instalación, la baja lógica y la generación de documentos PDF.

### 3.1. Alcance y entradas HTTP

- **Controlador:** ContratoMedidorController.
- **Servicio:** ContratoMedidorService.
- **Casos:** CreateContractUseCase, FindAllContractsUseCase, FindOneContractUseCase, UpdateContractUseCase, FinalizeMeterLinkUseCase, RemoveContractUseCase.
- La asignación de ruta usa el servicio/repositorios; no se confirmó un caso de uso dedicado.

| Método y ruta               | Caso/servicio ejecutado                                | Resultado             |
|-----------------------------|--------------------------------------------------------|-----------------------|
| POST /api/v1/contracts      | crearContrato()<br>CreateContractUseCase.execute()     | Contrato creado.      |
| GET /api/v1/contracts       | buscarContratos()<br>FindAllContractsUseCase.execute() | Página.               |
| GET /api/v1/contracts/:id   | buscarContrato()<br>FindOneContractUseCase.execute()   | Detalle.              |
| PATCH /api/v1/contracts/:id | actualizar()<br>UpdateContractUseCase.execute()        | Contrato actualizado. |

| Método y ruta                                             | Caso/servicio ejecutado                                               | Resultado                    |
|-----------------------------------------------------------|-----------------------------------------------------------------------|------------------------------|
| POST<br>/api/v1/contracts/:id/finalize                    | finalizarVinculo()<br>FinalizeMeterLinkUseCase.execute()              | Vínculo finalizado.          |
| POST<br>/api/v1/contracts/:id/assign-installation-route   | assignInstallationRoute()<br>AssignInstallationRouteUseCase.execute() | Ruta y orden de instalación. |
| DELETE<br>/api/v1/contracts/:id                           | eliminar()<br>RemoveContractUseCase.execute()                         | Soft delete.                 |
| GET<br>/api/v1/contracts/:id/pdf/connection-request       | generateConnectionRequestPdf()                                        | PDF.                         |
| GET<br>/api/v1/contracts/:id/pdf/responsibility-agreement | generateResponsibilityAgreementPdf()                                  | PDF.                         |

### 3.2. Ejemplo JSON

```
{
  "contratoId": "1",
  "clienteId": "10",
  "medidorId": "20",
  "estadoServicio": "PENDIENTE_PAGO",
  "estadoCobranza": "NO_APLICA"
}
```

| Campo                                    | Tipo   | Regla/uso                            |
|------------------------------------------|--------|--------------------------------------|
| contratoId, clienteId, medidorId         | string | BigInt serializado.                  |
| categoriaTarifaId, comunidadId, sectorId | number | Relaciones territoriales/tarifarias. |
| numeroGuia, direccionSuministro          | string | Datos de instalación.                |
| estadoServicio                           | enum   | Ciclo operativo del servicio.        |
| estadoCobranza                           | enum   | Situación de cobro.                  |

### 3.3. Estados

| Dimensión | Valores confirmados                                                 | Significado                                        |
|-----------|---------------------------------------------------------------------|----------------------------------------------------|
| Servicio  | PENDIENTE_PAGO, PENDIENTE_INSTALACION, ACTIVO, SUSPENDIDO, RETIRADO | Pago, instalación, operación, suspensión y retiro. |
| Cobranza  | NO_APLICA, AL_DIA, EN_MORA                                          | Sin cobro, corriente o vencido.                    |

### 3.4. Efectos y transacciones

La creación valida relaciones y escribe directamente contrato, historial y medidor. La prefactura se obtiene indirectamente mediante `SELECT generar_prefactura_instalacion(...)` dentro de la transacción cuando `estadoServicio=PENDIENTE_PAGO`; no se crea directamente desde Prisma.

### 3.4.1. Cadena POST y efectos

La cadena es `ContratoMedidorController.crear() → ContratoMedidorService.crearContrato() → CreateContractUseCase.execute() → PrismaContractRepository`. Fuentes: `backend/src/operations/c` (aprox. líneas 30-190), `backend/src/operations/contracts/infrastructure/repositories/prisma-contrato` (aprox. líneas 1-220) y `backend/src/operations/contracts/interfaces/dto/create-contrato-medidor.dt` (aprox. líneas 1-140). Se validan DTO, cliente, categoría, disponibilidad del medidor y coherencia del estado inicial.

| Superficie                                                               | Tipo      | Efecto                                                  |
|--------------------------------------------------------------------------|-----------|---------------------------------------------------------|
| contratos                                                                | Directo   | Inserta el vínculo con PENDIENTE_PAGO y NO_APLICA.      |
| historial_medidores                                                      | Directo   | Registra el vínculo inicial.                            |
| medidores                                                                | Directo   | Reserva/actualiza el estado.                            |
| Comprobante,<br>ComprobanteDetalle,<br>Prefacturas,<br>PrefacturaDetalle | Indirecto | Los crea la función SQL dentro de la misma transacción. |

La función está en `backend/prisma/migrations/20260822010000_add_codigo_sistema_rubro_and_update` (aprox. líneas 1-260). Selecciona el rubro por categoría y `codigo_sistema_rubro='INSTALACION'`. `codigo_sri LIKE 'SERV-GUIA-%'` es un criterio histórico/legacy, no vigente.

### 3.4.2. Fórmula de instalación

$\text{subtotal} = \text{precioUnitarioGuiaRemision} \times \text{cantidad}$ ,  $\text{iva} = \text{ROUND}(\text{subtotal} \times \text{tasaImpuesto}, 2)$ ,  $\text{total} = \text{subtotal} + \text{iva}$

Fuente: bloque `generar_prefactura_instalacion` de la migración SQL, aprox. líneas 80-220. `cantidad` es la cantidad facturada en el detalle del rubro. Los nombres `precioUnitarioGuiaRemision` y `tasaImpuesto` son conceptuales; los nombres reales SQL pueden diferir en Rubros y en la tarifa de impuesto.

| Tabla Prisma                           | Uso                              |
|----------------------------------------|----------------------------------|
| Contratos                              | Contrato, estados y soft delete. |
| HistorialMedidores                     | Historial del vínculo.           |
| Medidores                              | Medidor asociado.                |
| CategoriaTarifa, Clientes              | Relaciones principales.          |
| Prefacturas, PrefacturaDetalle, Rubros | Cargo de instalación.            |
| Rutas, OrdenTrabajo                    | Instalación asignada.            |

## 3.5. Gráfico de estados

```
stateDiagram-v2
    [*] --> PENDIENTE_PAGO
    PENDIENTE_PAGO --> PENDIENTE_INSTALACION: pago de instalación
    PENDIENTE_INSTALACION --> ACTIVO: actualización explícita de estado
    ACTIVO --> SUSPENDIDO
    SUSPENDIDO --> RETIRADO
    No se encontró reactivación desde RETIRADO.
```

### 3.5.1. Estado de cobranza

```
stateDiagram-v2
    [*] --> NO_APLICA
    NO_APLICA --> AL_DIA: normalización para servicio operativo
    NO_APLICA --> EN_MORA: proceso de corte, si corresponde
    AL_DIA --> EN_MORA: proceso de corte
    EN_MORA --> AL_DIA: proceso de corte
    No se confirmó una transición automática hacia el servicio.
```

## 3.6. Casos de uso

### 3.6.1. Caso: Crear contrato

**Descripción:** crea el contrato y prepara la instalación del medidor.

**HTTP y ruta:** POST /api/v1/contracts.

**Permiso:** contracts:create.

**Cadena:** ContratoMedidorController.crear() ContratoMedidorService.crearContrato()  
CreateContractUseCase.execute() repositorios.

**Errores relevantes:** 400 datos/relaciones inválidas; 401; 403; 404; medidor no disponible.

**Efectos:** crea contrato, historial y prefactura en las operaciones confirmadas por el caso.

**Prisma:** Contratos, HistorialMedidores, Medidores, Prefacturas, PrefacturaDetalle, Rubros, Clientes, CategoriaTarifa.

**Entrada:**

```
{
  "clienteId": "10",
  "medidorId": "20",
  "categoriaTarifaId": 1,
  "numeroGuia": "G-0001",
  "direccionSuministro": "Av. Amazonas 123",
  "comunidadId": 1,
  "sectorId": null
}
```

**Salida:**

```
{
  "contratoId": "1",
  "clienteId": "10",
  "medidorId": "20",
  "estadoServicio": "PENDIENTE_PAGO",
  "estadoCobranza": "NO_APLICA"
}
```

### 3.6.2. Caso: Listar contratos

**Descripción:** devuelve contratos filtrados y paginados.

**HTTP y ruta:** GET /api/v1/contracts.

**Permiso:** contracts:read.

**Cadena:** ContratoMedidorController.buscarContratos() ContratoMedidorService.buscarContratos  
FindAllContractsUseCase.execute() repositorio.

**Errores relevantes:** 400 query inválida; 401; 403.

**Efectos:** ninguno; consulta y DTO puros.

**Prisma:** Contratos, Clientes, Medidores, CategoriaTarifa.

**Entrada:** sin body; query del FilterContractsDto (paginación y filtros definidos por el DTO).

**Salida:**

```
{"data":[{"contratoId":"1","estadoServicio":"ACTIVO"}],"meta":{"page":1,"limit":10,"total":1,"totalPages":1}}
```

### 3.6.3. Caso: Obtener contrato

**Descripción:** consulta el detalle de un contrato.

**HTTP y ruta:** GET /api/v1/contracts/:id.

**Permiso:** contracts:read.

**Cadena:** ContratoMedidorController.buscarContrato() ContratoMedidorService.buscarContrato()  
FindOneContractUseCase.execute() repositorio.

**Errores relevantes:** 400; 401; 403; 404.

**Efectos:** ninguno; mapeo puro.

**Prisma:** Contratos, Clientes, Medidores, CategoriaTarifa, HistorialMedidores.

**Entrada:** sin body; path { "id": "1" }.

**Salida:**

```
{"contratoId":"1","clienteId":"10","estadoServicio":"ACTIVO","estadoCobranza":"AL_DIA","historialMedidores":[]}
```

### 3.6.4. Caso: Actualizar contrato

**Descripción:** actualiza estados, dirección, sector o reemplaza el medidor si se envía medidorId.

**HTTP y ruta:** PATCH /api/v1/contracts/:id.

**Permiso:** contracts:update.

**Cadena:** ContratoMedidorController.actualizarContrato() ContratoMedidorService.actualizar()  
UpdateContractUseCase.execute() repositorio.

**Errores relevantes:** 400; 401; 403; 404; conflicto de estado/relación.

**Efectos:** actualización; el reemplazo indicado se ejecuta transaccionalmente según el caso.

**Prisma:** Contratos, Medidores, HistorialMedidores.

**Entrada:** path { "id": "1" }; body { "direccionSuministro": "Calle Nueva 10" }.

**Salida:**

```
{"contratoId":"1","direccionSuministro":"Calle Nueva 10","estadoServicio":"ACTIVO"}
```

### 3.6.5. Caso: Finalizar vínculo de medidor

**Descripción:** finaliza el vínculo activo del contrato con su medidor.

**HTTP y ruta:** POST /api/v1/contracts/:id/finalize.

**Permiso:** contracts:update.

**Cadena:** ContratoMedidorController.finalizarVinculo() ContratoMedidorService.finalizarVinculo()  
FinalizeMeterLinkUseCase.execute() repositorio.

**Errores relevantes:** 400; 401; 403; 404.

**Efectos:** registra el cierre en el historial.

**Prisma:** HistorialMedidores, Contratos, Medidores.

**Entrada:** sin body; path { "id": "1" }.

**Salida:**

```
{"contratoId":"1","fechaFin":"2026-09-16"}
```

### 3.6.6. Caso: Asignar ruta de instalación

**Descripción:** asigna un contrato pendiente de instalación a una ruta existente o crea una ruta de instalación y su orden.

**HTTP y ruta:** POST /api/v1/contracts/:id/assign-installation-route.

**Permiso:** contracts:update.

**Cadena:** ContratoMedidorController.assignInstallationRoute() ContratoMedidorService.assignInstallationRoute() repositorios de rutas/órdenes.

**Errores relevantes:** 400; 401; 403; 404; ruta destino inválida.

**Efectos:** crea/reutiliza Rutas y crea OrdenTrabajo; no se confirma atomicidad de ambas escrituras.

**Prisma:** Rutas, OrdenTrabajo, Contratos, Medidores.

**Entrada:** path { "id": "1" }; body { "routeId": "42" } o DTO sin routeId según el código.

**Salida:**

```
{"rutaId":"42","tipoRuta":"INSTALACION","ordenesTrabajo":[{"ordenTrabajoId":"9","estado":"PENDIENTE"}]}
```

### 3.6.7. Caso: Eliminar contrato

**Descripción:** marca el contrato como eliminado lógicamente.

**HTTP y ruta:** DELETE /api/v1/contracts/:id.

**Permiso:** contracts:delete.

**Cadena:** ContratoMedidorController.eliminarContrato() ContratoMedidorService.eliminar() RemoveContractUseCase.execute() repositorio.

**Errores relevantes:** 400; 401; 403; 404.

**Efectos:** soft delete en Contratos.

**Prisma:** Contratos.

**Entrada:** sin body; path { "id": "1" }.

**Salida:**

```
{"contratoId":"1","estadoServicio":"ACTIVO","estadoCobranza":"AL_DIA"}
```

### 3.6.8. Caso: Generar PDF de solicitud de conexión

**Descripción:** genera la solicitud oficial de conexión.

**HTTP y ruta:** GET /api/v1/contracts/:id/pdf/connection-request.

**Permiso:** contracts:read.

**Cadena:** ContratoMedidorController.connectionRequestPdf() ContratoMedidorService.generateConnectionRequestPdf() GetConnectionRequestPdfDataUseCase generador.

**Errores relevantes:** 400; 401; 403; 404; aborto de solicitud.

**Efectos:** sólo lectura y proyección; serialización PDF pura.

**Prisma:** Contratos, Clientes, Medidores, CategoriaTarifa.

**Entrada:** sin body; path { "id": "1" }.

**Salida:** no es JSON: Content-Type: application/pdf, Content-Disposition: inline y Content-Length, con bytes PDF.

### 3.6.9. Caso: Generar PDF de acuerdo de responsabilidad

**Descripción:** genera el acta de responsabilidad asociada al contrato.

**HTTP y ruta:** GET /api/v1/contracts/:id/pdf/responsibility-agreement.

**Permiso:** contracts:read.

**Cadena:** `ContratoMedidorController.responsibilityAgreementPdf()` `ContratoMedidorService.gene`  
`GetResponsibilityAgreementPdfDataUseCase` `generador`.

**Errores relevantes:** 400; 401; 403; 404; aborto de solicitud.

**Efectos:** no se confirma escritura de dominio; proyección y PDF puros.

**Prisma:** Contratos, Clientes, Medidores, HistorialMedidores.

**Entrada:** sin body; path { "id": "1" }.

**Salida:** no es JSON: `application/pdf`, `Content-Disposition: inline`, `Content-Length` y bytes PDF.

### 3.7. No documentado o pendiente de confirmar



Figura 2: Contrato y prefactura de instalación.

- No se encontró transición automática a ACTIVO. `PagoValidadoHandler`, en `backend/src/billing/collection` (aprox. líneas 1-220), confirma `PENDIENTE_PAGO` → `PENDIENTE_INSTALACION` y cobranza `NO_APLICA`.
- Parámetros exactos de los DTO de finalización/asignación si cambian.
- Sincronización del campo `legacy estado`.

## 4. Convenios de pago

Financiación de deuda contractual, cuotas y efectos de pagos relacionados.

**Ficha de dominio:** documenta convenios, cuotas, resumen de deuda y pagos relacionados, incluidos sus estados, efectos derivados y superficies de generación de PDF.

### 4.1. Alcance y entradas HTTP

- **Controladores:** `AgreementsController` y, para pagos que afectan cuotas, `PaymentsController`.
- **Servicios:** `AgreementsService` y `PaymentsService`.
- **Casos:** `GetDebtSummaryUseCase`, `CreateAgreementUseCase`, `FindOneAgreementUseCase`, `UpdateAgreementUseCase`, `GetPaymentAgreementPdfDataUseCase`, `CreatePaymentUseCase`, `ValidatePaymentUseCase`, `AnnulPaymentUseCase`, `ApplySaldoFavorUseCase`.
- Listar convenios y catálogos se resuelven directamente en los servicios; no se confirmó caso dedicado para `findAll` ni para catálogos.

| Método y ruta                                      | Caso/servicio ejecutado                              | Resultado                           |
|----------------------------------------------------|------------------------------------------------------|-------------------------------------|
| GET<br>/api/v1/agreements/states                   | findAllAgreementStates()                             | Estados de convenio.                |
| GET<br>/api/v1/agreements/installment-states       | findAllInstallmentStates()                           | Estados de cuota.                   |
| GET<br>/api/v1/agreements/debt-summary             | getDebtSummary()<br><del>GetDebtSummaryUseCase</del> | Deuda.                              |
| POST /api/v1/agreements                            | create()<br>CreateAgreementUseCase                   | Convenio y cuotas.                  |
| GET /api/v1/agreements / :id<br>/ :id/installments | servicio /<br>FindOneAgreementUseCase                | Convenio/cuotas.                    |
| PATCH<br>/api/v1/agreements/:id                    | update()<br>UpdateAgreementUseCase                   | Estado.                             |
| DELETE<br>/api/v1/agreements/:id                   | cancel()<br>UpdateAgreementUseCase (ANULADO)         | Anulación.                          |
| GET<br>/api/v1/agreements/:id/pdf                  | generatePdf()<br>GetPaymentAgreementPdfDataUseCase   | PDF.                                |
| POST/PATCH/DELETE<br>/api/v1/payments...           | casos de pagos                                       | Crea, valida, aplica o anula pagos. |

## 4.2. Ejemplo JSON

```
{
  "convenioId": "8",
  "contratoId": "1",
  "deudaTotal": 240.5,
  "abonoInicial": 40,
  "numeroCuotas": 5,
  "estado": "PENDIENTE_ABONO",
  "cuotas": [
    {
      "cuotaConvenioId": "81",
      "numeroCuota": 1,
      "valorCuota": 40.1,
      "estado": "PENDIENTE"
    }
  ]
}
```

| Campo                                   | Tipo   | Regla/uso                                   |
|-----------------------------------------|--------|---------------------------------------------|
| convenioId, contratoId, cuotaConvenioId | string | BigInt serializado.                         |
| deudaTotal, abonoInicial, valorCuota    | number | Valores monetarios.                         |
| numeroCuotas, numeroCuota               | number | Plan y posición.                            |
| estado                                  | enum   | Estado de convenio o cuota según el objeto. |

## 4.3. Estados

| Entidad          | Estados                                             | Significado                                            |
|------------------|-----------------------------------------------------|--------------------------------------------------------|
| Convenio         | PREPARADO, PENDIENTE_ABONO, ACTIVO, PAGADO, ANULADO | Creado, abono pendiente, vigente, liquidado o anulado. |
| Cuota            | PENDIENTE, PAGADA                                   | No liquidada o liquidada.                              |
| Pago relacionado | PENDIENTE, REGISTRADO, ANULADO                      | Por validar, validado o anulado.                       |

## 4.4. Efectos y transacciones

Crear convenio calcula deuda y cuotas y persiste Convenios/CuotaConvenio en transacción. PagoValidadoHandler actualiza prefacturas y contratos de instalación; CuotaPagadaHandler verifica cuotas y dispara emisión SRI. PagoAnuladoHandler emite nota de crédito si corresponde; no se confirmó rollback de cuotas, prefacturas o contratos. La generación de catálogos, consultas y DTO es pura.

### 4.4.1. Tablas y fórmulas confirmadas

La relación es Contratos → Convenios → CuotaConvenio → PrefacturaDetalle → Prefacturas. La deuda usa prefacturas no eliminadas en estados GENERADA, EN\_REVISION o APROBADA:

$$\text{saldoPrefactura} = \max(0, \text{totalPagar} - \text{abono}), \quad \text{deudaTotal} = \text{ROUND\_HALF\_UP}(\sum \text{saldoPrefactura}, 2).$$

mesesMoraActual cuenta saldos positivos y deudaAnterior busca el período inmediatamente anterior; la continuidad de períodos es una limitación. Fuentes: backend/src/billing/collections/agreements/appl. prisma-agreement-repository.ts y backend/src/shared/utils/debt-calculator.util.ts.

La tasa se normaliza como tasaMensual=tasa/100; el interés lineal es

$$\text{interés} = (\text{deudaTotal} - \text{abonoInicial}) \times \text{tasaMensual} \times \text{numeroCuotas}, \quad \text{totalADistribuir} = \text{deudaTotal} - \text{abonoInicial} + \text{interés}$$

La cuota base es  $\text{TRUNC}(\text{totalADistribuir}/\text{numeroCuotas}, 2)$  y la última absorbe el residuo. CreateAgreementUseCase (aprox. líneas 30-220) valida deuda, abono, cuotas, tasa y transiciones en transacción. El vencimiento es mensual desde la fecha base.

| Tabla Prisma                                      | Uso                       |
|---------------------------------------------------|---------------------------|
| Convenios                                         | Plan y estado.            |
| CuotaConvenio                                     | Cuotas y pagos aplicados. |
| Contratos, Prefacturas, PrefacturaDetalle, Rubros | Deuda de origen.          |
| Pagos, DetallePago, SaldoFavor                    | Cobros y saldos.          |

## 4.5. Transiciones de estado del convenio

### 4.5.1. Cadena de transiciones confirmada

| De A                         | Cuándo se ejecuta                        | Implementación                       |
|------------------------------|------------------------------------------|--------------------------------------|
| (creación) → PREPARADO       | POST /api/v1/agreements sin abonoInicial | CreateAgreementUseCase (transacción) |
| (creación) → PENDIENTE_ABONO | POST /api/v1/agreements con abonoInicial | mismo                                |

| De A                                  | Cuándo se ejecuta                                 | Implementación                   |
|---------------------------------------|---------------------------------------------------|----------------------------------|
| PREPARADO o PENDIENTE_ABONO → ACTIVO  | PATCH<br>/api/v1/agreements/:id con estado ACTIVO | UpdateAgreementUseCase.execute(A |
| PREPARADO o PENDIENTE_ABONO → PAGADO  | PATCH<br>/api/v1/agreements/:id con estado PAGADO | agreementRepository.markAsPaid   |
| PREPARADO o PENDIENTE_ABONO → ANULADO | DELETE<br>/api/v1/agreements/:id                  | UpdateAgreementUseCase.execute(A |
| ACTIVO → PAGADO                       | mismo PATCH                                       | mismo camino                     |
| ACTIVO → ANULADO                      | mismo DELETE                                      | mismo camino                     |
| PAGADO → <i>cualquiera</i>            | <b>bloqueado</b>                                  | throw<br>BadRequestException     |
| ANULADO → <i>cualquiera</i>           | <b>bloqueado</b>                                  | throw<br>BadRequestException     |

#### 4.5.2. Handlers de efectos derivados

Los pagos contra cuotas no cambian el estado del convenio directamente: disparan handlers que emiten comprobantes SRI.

| Handler             | Disparo                       | Efecto confirmado                                                                                                                       |
|---------------------|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| PagoValidadoHandler | evento pago.validado (outbox) | marca prefactura PAGADA y promueve contrato a PENDIENTE_INSTALACION                                                                     |
| CuotaPagadaHandler  | evento cuota.pagada (outbox)  | <b>verifica</b> que todas las cuotas estén PAGADA y entonces dispara emisión SRI del comprobante. <b>No marca la cuota como PAGADA.</b> |
| PagoAnuladoHandler  | evento pago.anulado (outbox)  | emite Nota de Crédito solo si el comprobante está AUTORIZADO en SRI. Si está en otro estado, no emite.                                  |

#### 4.5.3. Restricciones operativas

- PAGADO y ANULADO son **terminales**. Cualquier intento de transición desde ellos falla con BadRequestException (update-agreement.use-case.ts:36-39).
- PAGADO se asigna solo cuando todas las cuotas están PAGADAS (markAsPaid en el repositorio). El camino normal es que la última cuota llegue a saldo 0 por acumulación de pagos y dispare el handler; el PATCH explícito a PAGADO es un atajo administrativo.
- Cuota PAGADA se calcula por saldo, no por evento. La transición cuota.estado: PENDIENTE PAGADA ocurre en apply-saldo-favor.use-case.ts:135-142 cuando saldoPendiente.equals(0). Cada aplicación de saldo re-evalúa.
- Outbox pattern en pagos. ValidatePaymentUseCase no ejecuta la transición directamente; persiste un evento en eventos\_pendientes que un dispatcher consume para invocar al handler.

#### 4.6. Gráfico de estados

```
stateDiagram-v2
    [*] --> PREPARADO: crear sin abono inicial
    [*] --> PENDIENTE_ABONO: crear con abono inicial
```

```

PREPARADO --> ACTIVO: PATCH estado=ACTIVO
PENDIENTE_ABONO --> ACTIVO: PATCH estado=ACTIVO
PREPARADO --> PAGADO: PATCH estado=PAGADO
PENDIENTE_ABONO --> PAGADO: PATCH estado=PAGADO
ACTIVO --> PAGADO: PATCH estado=PAGADO
PREPARADO --> ANULADO: DELETE
PENDIENTE_ABONO --> ANULADO: DELETE
ACTIVO --> ANULADO: DELETE
No se encontraron transiciones salientes confirmadas desde PAGADO o ANULADO.

```

#### 4.6.1. Estado de la cuota

```

stateDiagram-v2
    [*] --> PENDIENTE
    PENDIENTE --> PAGADA: CuotaPagadaHandler
    No se encontró una reactivación PAGADA --> PENDIENTE.

```

### 4.7. Casos de uso

#### 4.7.1. Caso: Consultar estados de convenio

**Descripción:** entrega el catálogo de estados disponibles para convenios.

**HTTP y ruta:** GET /api/v1/agreements/states.

**Permiso:** agreements:read.

**Cadena:** `AgreementsController.findAllStates()` `AgreementsService.findAllAgreementStates()` enum.

**Errores relevantes:** 401; 403.

**Efectos:** ninguno; catálogo de lectura.

**Prisma:** ninguna tabla.

**Entrada:** sin body, path ni query.

**Salida:** [ { "codigo": "ACTIVO", "descripcion": "Activo" } ].

#### 4.7.2. Caso: Consultar estados de cuota

**Descripción:** entrega el catálogo de estados disponibles para cuotas.

**HTTP y ruta:** GET /api/v1/agreements/installment-states.

**Permiso:** agreements:read.

**Cadena:** `AgreementsController.findAllInstallmentStates()` `AgreementsService.findAllInstallmentStates()` enum.

**Errores relevantes:** 401; 403.

**Efectos:** ninguno; catálogo de lectura.

**Prisma:** ninguna tabla.

**Entrada:** sin body, path ni query.

**Salida:** [ { "codigo": "PENDIENTE", "descripcion": "Pendiente" } ].

#### 4.7.3. Caso: Consultar resumen de deuda

**Descripción:** calcula el resumen de deuda pendiente y devuelve el detalle de las prefacturas impagadas. No expone un total agregado montoPagado en este caso de uso.

**HTTP y ruta:** GET /api/v1/agreements/debt-summary/:contratoId.

**Permiso:** agreements:read.

**Cadena:** `AgreementsController.getDebtSummary()` `AgreementsService.getDebtSummary()` `GetDebtSummaryUseCase.execute()` repositorio.

**Errores relevantes:** 400; 401; 403; 404.

**Efectos:** ninguno.

**Prisma:** Contratos, Prefacturas, PrefacturaDetalle, Rubros, pagos.

**Entrada:** sin body; path { "contratoId": "1" }.

**Salida:** { "contratoId": "1", "deudaTotal": 240.5, "deudaAnterior": 40, "tasaMensualVigente": 2.5, "maxMesesAtrasado": 2, "totalPrefacturasImpagadas": 3, "prefacturas": [] }.

#### 4.7.4. Caso: Crear convenio

**Descripción:** crea un plan de pago y genera sus cuotas.

**HTTP y ruta:** POST /api/v1/agreements.

**Permiso:** agreements:create.

**Cadena:** AgreementsController.create() AgreementsService.create() CreateAgreementUseCase.ex  
repositorio.

**Errores relevantes:** 400; 401; 403; 404; deuda no conveniente.

**Efectos:** persiste convenio y cuotas; no activa por sí solo el contrato.

**Prisma:** Convenios, CuotaConvenio, Contratos, prefacturas/rubros.

**Entrada:** { "contratoId": "1", "abonoInicial": 40, "numeroCuotas": 5 }.

**Salida:** JSON del ejemplo principal.

#### 4.7.5. Caso: Listar convenios

**Descripción:** lista convenios paginados y filtrables.

**HTTP y ruta:** GET /api/v1/agreements.

**Permiso:** agreements:read.

**Cadena:** AgreementsController.findAll() AgreementsService.findAll() repositorio;  
sin caso dedicado confirmado.

**Errores relevantes:** 400; 401; 403.

**Efectos:** ninguno.

**Prisma:** Convenios, contrato/cliente.

**Entrada:** sin body; query page, limit, contratoId, estado, search.

**Salida:** { "data": [{ "convenioId": "8", "estado": "ACTIVO" }], "meta": {} }.

#### 4.7.6. Caso: Obtener convenio

**Descripción:** devuelve el detalle de un convenio por identificador.

**HTTP y ruta:** GET /api/v1/agreements/:id.

**Permiso:** agreements:read.

**Cadena:** AgreementsController.findOne() AgreementsService.findOne() FindOneAgreementUseCase.ex  
repositorio.

**Errores relevantes:** 400; 401; 403; 404.

**Efectos:** ninguno; consulta y DTO puros.

**Prisma:** Convenios, Contratos, Clientes, CuotaConvenio.

**Entrada:** sin body; path { "id": "8" }.

**Salida:** objeto JSON AgreementResponseDto con sus datos y cuotas.

#### 4.7.7. Caso: Listar cuotas de un convenio

**Descripción:** devuelve las cuotas asociadas a un convenio.

**HTTP y ruta:** GET /api/v1/agreements/:id/installments.

**Permiso:** agreements:read.

**Cadena:** `AgreementsController.findInstallments() AgreementsService.findInstallments()` repositorio.

**Errores relevantes:** 400; 401; 403; 404.

**Efectos:** ninguno; consulta y DTO puros.

**Prisma:** `CuotaConvenio, Convenios`.

**Entrada:** sin body; path { "id": "8" }.

**Salida:** lista JSON [ { "cuotaConvenioId": "81", "numeroCuota": 1, "estado": "PENDIENTE" } ].

#### 4.7.8. Caso: Actualizar estado del convenio

**Descripción:** aprueba, paga o cambia el estado permitido del convenio.

**HTTP y ruta:** `PATCH /api/v1/agreements/:id`.

**Permiso:** `agreements:update`.

**Cadena:** `AgreementsController.update() AgreementsService.update() UpdateAgreementUseCase.execute()` repositorio.

**Errores relevantes:** 400; 401; 403; 404; transición inválida.

**Efectos:** PAGADO puede marcar convenio y cuotas.

**Prisma:** `Convenios, CuotaConvenio`.

**Entrada:** path { "id": "8" }; body { "estado": "ACTIVO" }.

**Salida:** { "convenioId": "8", "estado": "ACTIVO" }.

#### 4.7.9. Caso: Anular convenio

**Descripción:** marca un convenio como ANULADO.

**HTTP y ruta:** `DELETE /api/v1/agreements/:id`.

**Permiso:** `agreements:delete`.

**Cadena:** `AgreementsController.cancel() AgreementsService.cancel() UpdateAgreementUseCase(AN`

**Errores relevantes:** 400; 401; 403; 404.

**Efectos:** persiste anulación; no se confirma borrado físico.

**Prisma:** `Convenios, cuotas` si la transición las afecta.

**Entrada:** sin body; path { "id": "8" }.

**Salida:** { "convenioId": "8", "estado": "ANULADO" }.

#### 4.7.10. Caso: Generar PDF del convenio

**Descripción:** genera el documento imprimible del plan.

**HTTP y ruta:** `GET /api/v1/agreements/:id/pdf`.

**Permiso:** `agreements:read`.

**Cadena:** `AgreementsController.generatePdf() AgreementsService.generatePdf() GetPaymentAgreementPdfDataUseCase.execute() dispatcher`.

**Errores relevantes:** 400; 401; 403; 404; aborto de solicitud.

**Efectos:** sólo lectura y proyección; PDF puro.

**Prisma:** `Convenios, CuotaConvenio, Contratos, Clientes`.

**Entrada:** sin body; path { "id": "8" }.

**Salida:** no es JSON: `application/pdf, Content-Disposition: inline, Content-Length` y bytes PDF.

### 4.8. No documentado o pendiente de confirmar

**Pendiente de confirmar contra código actual:**



Figura 3: Convenio y pago.

- Forma exacta de los cuerpos de request/response para POST `/api/v1/agreements`, PATCH `/api/v1/agreements/:id` y DELETE `/api/v1/agreements/:id`. Los DTOs pueden haber cambiado desde la última revisión.
- Forma exacta de `cuotaConvenioId` en `apply-saldo-favor` (parámetro que dispara la transición de cuota). Confirmar contra el DTO actual.
- Handler explícito para la transición `cuota.estado: PENDIENTE VENCIDA`. Hoy no se encontró un handler ni un job programado que marque cuotas vencidas; el documento decía “Umbral que convierte una cuota vencida en causal automática de corte” pero no hay implementación rastreable en el código revisado.
- Política completa de mora: el documento decía “No se encontró una política completa de mora”. Sigue pendiente; el ciclo de cutoff (`collection-cutoff.service.ts`) opera a nivel de prefacturas, no de cuotas individuales.

## 5. Rutas y órdenes de trabajo

Despacho de lecturas, instalaciones y operación de campo.

**Ficha de dominio:** documenta períodos, rutas, órdenes de trabajo y operaciones del personal de campo, incluida la vinculación de lecturas y la evidencia externa.

### 5.1. Alcance y entradas HTTP

- **Controladores:** `RoutesController`, `OrdenesTrabajoController`, `OperatorController`.
- **Servicios:** `RoutesService`, `OrdenesTrabajoService`.
- **Casos:** `CreateRouteUseCase`, `FindAllRoutesUseCase`, `FindOneRouteUseCase`, `UpdateRouteUseCase`, `DeleteRouteUseCase`, `ReassignRouteUseCase`, `GetEligibleReadingsUseCase`, `GetReadingsByRutaUseCase`, `FindOrdenesByRutaUseCase`, `ExportFieldSheetPdfUseCase`, `UpdateOrdenEstadoUseCase`, `LinkLecturaUseCase`, `GetOperatorRoutesUseCase`, `UpdateRouteStateUseCase`, `UpdateOperatorWorkOrderUseCase`.
- Los períodos se consultan directamente desde `RoutesService`; no se confirmó caso dedicado para listar rutas administrativas ni para algunos cambios de órdenes.

## 5.2. Comportamiento de negocio verificable

Las tablas relacionadas son Rutas, OrdenTrabajo, EjecucionOrdenTrabajo, Lecturas, Periodos y NovedadOrdenTrabajo. El flujo confirmado es período → ruta → orden → lectura/evidencia → cierre. No se encontró fórmula monetaria que aplique a este módulo. La promoción automática del contrato a ACTIVO al completar orden de instalación sí está implementada — ver 02-contratos.md, sección “Transiciones de estado del contrato”.

| Método y ruta                          | Caso/servicio ejecutado                               | Resultado                     |
|----------------------------------------|-------------------------------------------------------|-------------------------------|
| GET /api/v1/routes/periods             | RoutesService.getPeriodos()                           | Períodos.                     |
| GET /api/v1/routes/eligible-readings   | getEligibleReadings()<br>GetEligibleReadingsUseCase   | Lecturas elegibles paginadas. |
| GET /api/v1/routes/:rutaId/readings    | getReadingsByRuta()<br>GetReadingsByRutaUseCase       | Lecturas de ruta.             |
| POST /api/v1/routes                    | create() CreateRouteUseCase                           | Ruta creada.                  |
| GET /api/v1/routes/:id                 | findAll() / findOne() casos de consulta               | Rutas.                        |
| PATCH /api/v1/routes/:id               | update() UpdateRouteUseCase                           | Ruta actualizada.             |
| PATCH /api/v1/routes/:id/reassign      | ReassignRouteUseCase.execute()                        | Operario reasignado.          |
| DELETE /api/v1/routes/:id              | delete() DeleteRouteUseCase                           | Soft delete.                  |
| GET /api/v1/routes/:id/pdf             | ExportFieldSheetPdfUseCase                            | PDF.                          |
| GET /api/v1/routes/:id/work-orders     | FindOrdenesByRutaUseCase                              | Órdenes paginadas.            |
| PATCH /api/v1/work-orders/:id/state    | UpdateOrdenEstadoUseCase                              | Estado de orden.              |
| PATCH /api/v1/work-orders/:id/reading  | LinkLecturaUseCase                                    | Lectura vinculada.            |
| GET/PATCH /api/v1/operator/routes...   | GetOperatorRoutesUseCase /<br>UpdateRouteStateUseCase | Vista y estado del operador.  |
| PATCH /api/v1/operator/work-orders/:id | UpdateOperatorWorkOrderUseCase                        | Ejecución/evidencia.          |

## 5.3. Ejemplo JSON

```
{
  "rutaId": "42",
  "nombre": "Lecturas agosto",
  "tipoRuta": "LECTURA",
  "estado": "PENDIENTE",
  "ordenesTrabajo": []
}
```

| Campo                              | Tipo        | Regla/uso                            |
|------------------------------------|-------------|--------------------------------------|
| rutaId, ordenTrabajoId, lecturaId  | string      | BigInt serializado.                  |
| tipoRuta                           | enum        | Actividad de la ruta.                |
| estado                             | enum        | Estado de ruta u orden según el DTO. |
| operarioId, periodoId, comunidadId | number/null | Asignación y contexto.               |

| Campo          | Tipo  | Regla/uso          |
|----------------|-------|--------------------|
| ordenesTrabajo | array | Órdenes asociadas. |

## 5.4. Estados

| Entidad | Estados confirmados                                    | Significado                                               |
|---------|--------------------------------------------------------|-----------------------------------------------------------|
| Ruta    | PENDIENTE, EN_PROGRESO, COMPLETADA, PARCIAL, CANCELADA | Pendiente, ejecutada, finalizada, incompleta o cancelada. |
| Orden   | PENDIENTE, EN_PROGRESO, COMPLETADA, CANCELADA, FALLIDA | Ciclo de trabajo.                                         |

### 5.4.1. Máquina de estados de la Ruta

La máquina está implementada como una tabla de transiciones explícita en `backend/src/operations/routes/dor`

| Desde       | Hacia permitidas                               |
|-------------|------------------------------------------------|
| PENDIENTE   | EN_PROGRESO, CANCELADA                         |
| EN_PROGRESO | COMPLETADA, PARCIAL, PENDIENTE, CANCELADA      |
| COMPLETADA  | EN_PROGRESO (reabre para corrección de cierre) |
| PARCIAL     | EN_PROGRESO (reabre para continuar trabajo)    |
| CANCELADA   | PENDIENTE (reabre para reactivación)           |

La función `canTransitionRouteState(from, to)` valida la transición; `UpdateRouteUseCase.execute()` rechaza con `InvalidDomainOperationException` si la transición no está permitida. **No existe máquina equivalente para OrdenTrabajo** — sus transiciones las acepta todas `UpdateOrdenEstadoUseCase` mientras el estado pertenezca al enum.

## 5.5. Regla de novedades sin estado adicional de orden

**Decisión funcional propuesta pendiente de implementación.** No se agregará un estado persistente NOVEDAD a `OrdenTrabajo`. Mientras exista una novedad, la orden permanece en `EN_PROGRESO`; la interfaz muestra una alerta derivada al consultar la relación de novedades activas.

### 5.5.1. Flujo acordado de orden con novedad

Resumen textual del flujo:

- La orden inicia en `PENDIENTE`; el operador la pasa a `EN_PROGRESO` al iniciar el trabajo.
- Sin novedad, la orden se completa (`COMPLETADA`).
- Con novedad en una orden que NO es de lectura, la orden queda en `EN_PROGRESO` con `tieneNovedadActiva=true` hasta resolverla; si el caso es irre recuperable pasa a `FALLIDA`, y si se cancela pasa a `CANCELADA`.
- En una orden de `LECTURA` con medidor dañado, el operador crea una `NovedadOrdenTrabajo` en `OPEN`; la lectura queda en `CON_NOVEDAD` y la orden permanece en `EN_PROGRESO` bloqueada.

## Flujo acordado de Orden, Lectura y Novedad — medidor dañado y estimación

Política acordada: la autorización de estimación por secretaría y el bloqueo por novedad están pendientes de implementación.

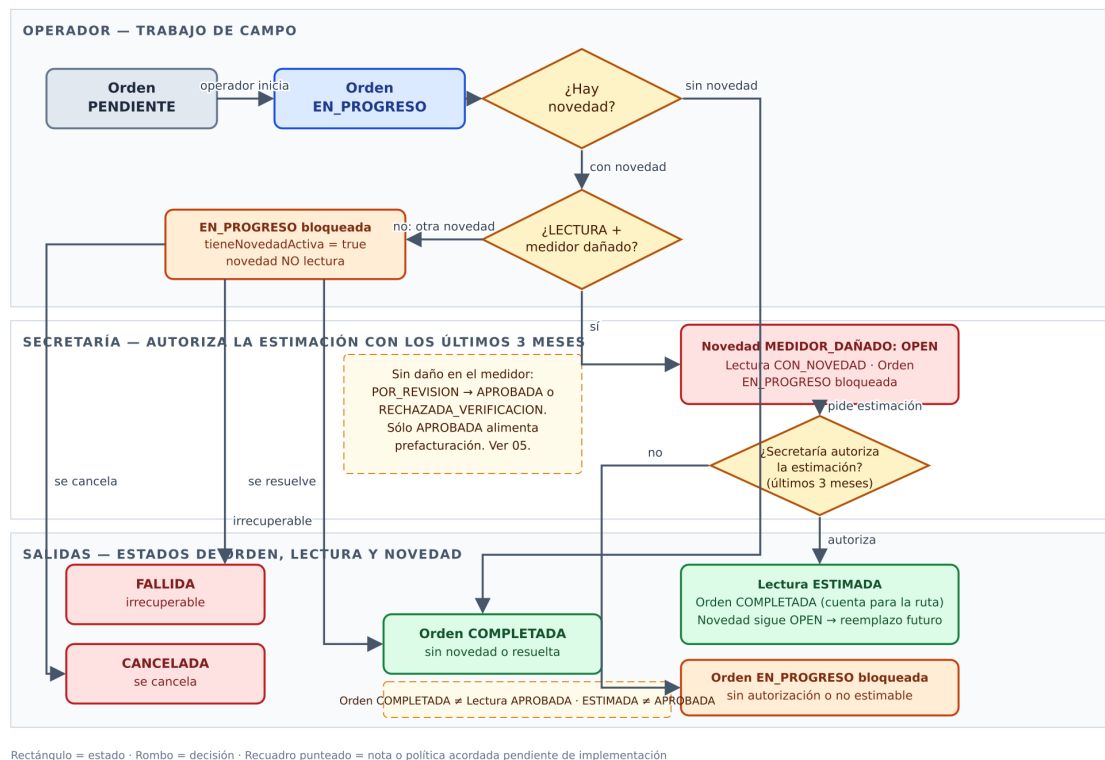


Figura 4: Flujo acordado de orden, lectura y novedad. Fuente editable: diagrams/orden-lectura-novedad.drawio.

- Sólo secretaría puede autorizar la estimación con los últimos 3 meses (**política acordada, pendiente de implementación**): si autoriza, la lectura pasa a ESTIMADA y la orden a COMPLETADA, mientras la novedad sigue en OPEN para un reemplazo futuro; si no autoriza o no se puede estimar, la orden sigue bloqueada.
- La orden COMPLETADA no implica lectura APROBADA: la estimación genera ESTIMADA, no APROBADA.
- La orden completada por estimación autorizada cuenta como COMPLETADA para el cierre de la ruta.

NOVEDAD no es un valor del enum de OrdenTrabajo: es una condición derivada de la relación con NovedadOrdenTrabajo y de la existencia de novedades activas. No debe persistirse ni confundirse con OrdenTrabajo.estado.

### 5.5.2. Tres máquinas separadas

| Entidad                    | Máquina                                                                      | Responsabilidad                                    |
|----------------------------|------------------------------------------------------------------------------|----------------------------------------------------|
| OrdenTrabajo.estado        | Ejecución operativa: PENDIENTE, EN_PROGRESO, COMPLETADA, CANCELADA, FALLIDA. | Etapa del trabajo de campo y cierre de la orden.   |
| NovedadOrdenTrabajo.estado | Análisis de anomalía: OPEN, IN_PROGRESS, RESOLVED, CANCELLED.                | Atención, resolución o cancelación de la anomalía. |

| Entidad        | Máquina                                                                                                             | Responsabilidad                                                         |
|----------------|---------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| Lectura.estado | Captura y validación: PENDIENTE, POR_REVISION, APROBADA, RECHAZADA_VERIFICACION, ESTIMADA, PLANILLADA, CON_NOVEDAD. | Ciclo de captura, revisión, aprobación, rechazo o tratamiento del dato. |

Una novedad activa puede generar una alerta sobre la orden, pero no transforma su estado persistido en NOVEDAD; una lectura con CON\_NOVEDAD tampoco reemplaza el estado de la orden ni el estado de la novedad.

### 5.5.3. Ejemplo de respuesta derivada

```
{
  "ordenTrabajoId": "9",
  "estado": "EN_PROGRESO",
  "tieneNovedadActiva": true,
  "novedadesActivas": [
    {
      "novedadOrdenTrabajoId": "31",
      "estado": "OPEN",
      "tipo": "LECTURA_ERRONEA",
      "observacion": "El visor no permite validar el consumo"
    }
  ]
}
```

### 5.5.4. Regla de backend y pendiente funcional

La UI no es la única barrera. Como regla de backend propuesta, una operación no debe permitir pasar la orden a COMPLETADA si existe una novedad abierta que afecte el resultado; la validación debe ejecutarse en el caso de uso o repositorio que completa la orden. Esta regla está pendiente de implementación.

También está pendiente definir el campo o criterio que determina si una novedad es bloqueante: podría ser afectaOrden, el tipo de novedad, la resolución registrada o una regla equivalente. Hasta acordarlo, esta sección documenta una decisión funcional propuesta y no un comportamiento vigente.

También están pendientes de implementación la autorización de la estimación sólo por secretaría con los últimos 3 meses y el conteo de órdenes para el cierre COMPLETADA/PARCIAL de la ruta.

## 5.6. Efectos y transacciones

Crear/actualizar/eliminar rutas y órdenes escriben Rutas, OrdenTrabajo y, cuando corresponde, EjecucionOrdenTrabajo. Vincular una lectura completa la orden y fija completadoEn; cambiarla a pendiente/en progreso lo limpia. El operador puede almacenar evidencia en RustFS y su clave en la orden. Consultas, filtros, mapeadores y generación PDF son transformaciones puras; la subida de evidencia no lo es.

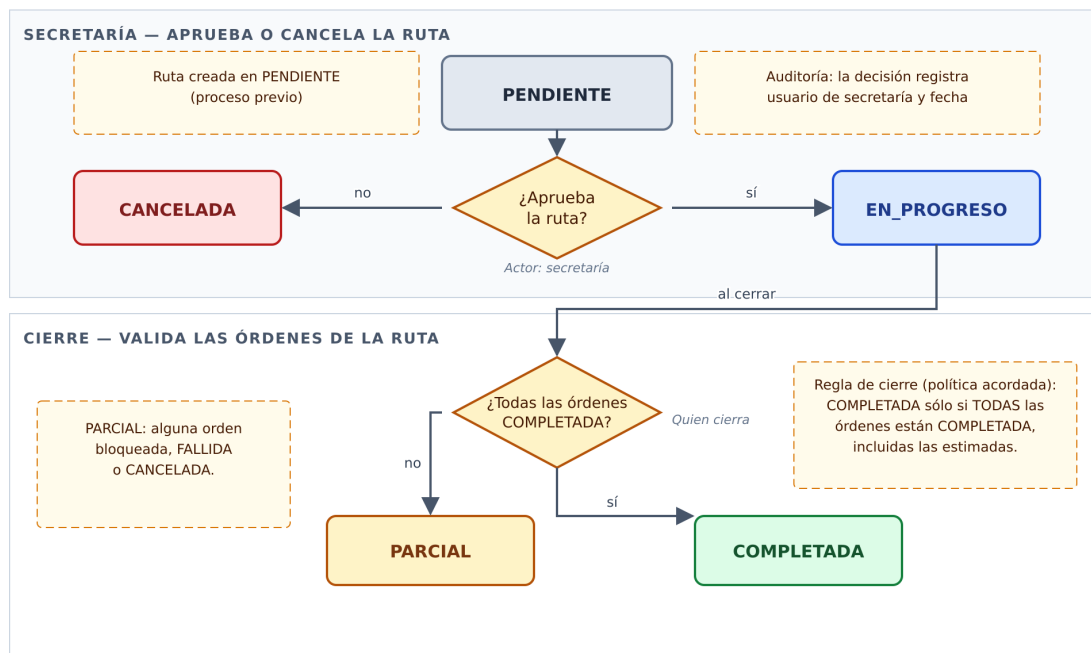
| Tabla Prisma          | Uso                                     |
|-----------------------|-----------------------------------------|
| Rutas                 | Ruta, asignación, estado y soft delete. |
| OrdenTrabajo          | Trabajo y vínculo con lectura.          |
| EjecucionOrdenTrabajo | Resultado de ejecución.                 |
| Lecturas              | Lecturas elegibles/vinculadas.          |

| Tabla Prisma        | Uso                 |
|---------------------|---------------------|
| Periodos            | Ciclo contable.     |
| NovedadOrdenTrabajo | Novedades de campo. |

## 5.7. Flujos acordados y máquinas de estado

### Flujo acordado de Ruta — aprobación de secretaría y cierre por órdenes

La orden estimada con autorización cuenta como COMPLETADA para la ruta. El conteo automático del cierre está pendiente de implementación.



Rectángulo = estado · Rombo = decisión · Recuadro punteado = nota o política acordada pendiente de implementación

Figura 5: Flujo acordado de la ruta. Fuente editable: `diagrams/ruta-estados.drawio`.

Resumen textual del flujo:

- La ruta nace en PENDIENTE.
- Secretaría decide con auditoría (usuario y fecha): si no la aprueba, la ruta pasa a CANCELADA; si la aprueba, pasa a EN\_PROGRESO.
- Al cerrar una ruta en EN\_PROGRESO se validan sus órdenes (**conteo pendiente de implementación**): si TODAS están COMPLETADA incluidas las completadas por estimación autorizada, la ruta pasa a COMPLETADA; si alguna sigue bloqueada, o está FALLIDA o CANCELADA, la ruta pasa a PARCIAL.

Alcance en código: `UpdateRouteStateUseCase` confirma las transiciones del operador. COMPLETADA y CANCELADA son terminales en ese caso de uso.

#### 5.7.1. Estado de la orden

Alcance: transiciones **acordadas** (política); los estados del enum y las escrituras están confirmados, pero `UpdateOrdenEstadoUseCase` no define una tabla de transiciones en código.

- PENDIENTE → EN\_PROGRESO: el operador inicia el trabajo.
- EN\_PROGRESO → COMPLETADA: sin novedad activa, con la novedad resuelta, o con estimación autorizada por secretaría.

- EN\_PROGRESO → FALLIDA: el caso es irrecuperable.
- EN\_PROGRESO → CANCELADA: se cancela el trabajo.
- Con novedad activa sin resolver, la orden permanece en EN\_PROGRESO con `tieneNovedadActiva=true` (ver flujo acordado).

### 5.7.2. Estado de las novedades de órdenes de trabajo

Sí existe una máquina de estados independiente para `NovedadOrdenTrabajo`. El modelo Prisma confirma OPEN, IN\_PROGRESS, RESOLVED y CANCELLED. La novedad se relaciona con una orden y opcionalmente con una lectura y el usuario que la resolvió.

Alcance: **Confirmado** para los estados, el valor inicial OPEN y los datos de resolución. No se encontró un controlador o caso de uso dedicado que confirme las transiciones HTTP; las flechas de atención y resolución son un flujo esperado.

La máquina, en texto:

- OPEN (valor inicial al crear la novedad) → IN\_PROGRESS: inicia la atención (flujo esperado).
- IN\_PROGRESS → RESOLVED: registra la resolución con usuario y fecha (flujo esperado).
- OPEN o IN\_PROGRESS → CANCELLED: se cancela (flujo esperado, sin reactivación confirmada).

La novedad de medidor dañado puede seguir en OPEN después de una estimación autorizada, para gestionar el reemplazo futuro del medidor.

| Elemento             | Valores o datos confirmados                                                               |
|----------------------|-------------------------------------------------------------------------------------------|
| Estado               | OPEN, IN_PROGRESS, RESOLVED, CANCELLED.                                                   |
| Tipo de anomalía     | FUGA, MEDIDOR_DAÑADO, LECTURA_ERRONEA, OTRO.                                              |
| Resolución económica | COBRO_REAL, PROMEDIO_HISTORICO, EXONERACION_PARCIAL, EXONERACION_TOTAL, AJUSTE_LECTURA.   |
| Datos de resolución  | resolucionTipo, consumoAjustado, observacionResolucion, resueltoPorUsuarioId, resueltoEn. |

`resultadoObservacion` y `estado=FALLIDA` de `OrdenesTrabajo` no sustituyen a `NovedadOrdenTrabajo`: representan el resultado de la orden, mientras que la novedad conserva la anomalía y su resolución.

## 5.8. Casos de uso

### 5.8.1. Caso: Consultar períodos

**Descripción:** lista períodos disponibles para filtros y asignación.

**HTTP y ruta:** GET /api/v1/routes/periods.

**Permiso:** routes:read.

**Cadena:** `RoutesController.getPeriods()` `RoutesService.getPeriodos()` repositorio.

**Errores relevantes:** 401; 403.

**Efectos:** ninguno.

**Prisma:** Periodos.

**Entrada:** sin body ni parámetros.

**Salida:**

```
[{"periodoId":12,"nombre":"2026-08","estado":"ACTIVO"}]
```

### 5.8.2. Caso: Consultar lecturas elegibles

**Descripción:** devuelve lecturas disponibles para asignarlas a una ruta.

**HTTP y ruta:** GET /api/v1/routes/eligible-readings.

**Permiso:** routes:read.

**Cadena:** RoutesController.getEligibleReadings() RoutesService.getEligibleReadings()  
GetEligibleReadingsUseCase.execute().

**Errores relevantes:** 400; 401; 403.

**Efectos:** ninguno.

**Prisma:** Lecturas, Contratos, Medidores, Periodos, OrdenTrabajo.

**Entrada:** sin body; query page, limit y filtros de FilterReadingsDto.

**Salida:**

```
{"data": [{"lecturaId": "101", "estado": "PENDIENTE"}], "meta": {"page": 1, "limit": 10, "total": 1, "totalPages": 1}, "kpis": {}}
```

### 5.8.3. Caso: Consultar lecturas de ruta

**Descripción:** lista lecturas ya vinculadas a una ruta.

**HTTP y ruta:** GET /api/v1/routes/:rutaId/readings.

**Permiso:** routes:read.

**Cadena:** RoutesController.getReadingsByRuta() RoutesService.getReadingsByRuta()  
GetReadingsByRutaUseCase.execute().

**Errores relevantes:** 400; 401; 403; 404.

**Efectos:** ninguno.

**Prisma:** Rutas, OrdenTrabajo, Lecturas.

**Entrada:** sin body; path { "rutaId": "42" }; query page, limit.

**Salida:**

```
{"data": [{"lecturaId": "101", "estado": "PENDIENTE"}], "meta": {"page": 1, "limit": 10, "total": 1, "totalPages": 1}, "kpis": {}}
```

### 5.8.4. Caso: Crear ruta

**Descripción:** crea una ruta de trabajo con su período y tipo.

**HTTP y ruta:** POST /api/v1/routes.

**Permiso:** routes:create.

**Cadena:** RoutesController.create() RoutesService.create() CreateRouteUseCase.execute()  
repositorio.

**Errores relevantes:** 400; 401; 403; 404 período/guías.

**Efectos:** alta en Rutas y asociaciones que determine el caso.

**Prisma:** Rutas, OrdenTrabajo, Contratos, Periodos.

**Entrada:** body { "nombre": "Lecturas agosto", "tipoRuta": "LECTURA", "periodoId":  
12 }.

**Salida:**

```
{"rutaId": "42", "nombre": "Lecturas agosto", "estado": "PENDIENTE", "ordenesTrabajo": []}
```

### 5.8.5. Caso: Listar rutas

**Descripción:** devuelve rutas con paginación y filtros.

**HTTP y ruta:** GET /api/v1/routes.

**Permiso:** routes:read.

**Cadena:** RoutesController.findAll() RoutesService.findAll() repositorio.

**Errores relevantes:** 400; 401; 403.

**Efectos:** ninguno.

**Prisma:** Rutas y relaciones.

**Entrada:** sin body; query page, limit, estado, operarioId, comunidadId, periodoId, tipoRuta.

**Salida:**

```
{"data": [{"rutaId": "42", "estado": "PENDIENTE"}], "meta": {"page": 1, "limit": 10, "total": 1, "totalPages": 1}}
```

### 5.8.6. Caso: Obtener ruta

**Descripción:** consulta una ruta por ID.

**HTTP y ruta:** GET /api/v1/routes/:id.

**Permiso:** routes:read.

**Cadena:** RoutesController.findOne() RoutesService.findOne() repositorio.

**Errores relevantes:** 400; 401; 403; 404.

**Efectos:** ninguno.

**Prisma:** Rutas, OrdenTrabajo, Contratos, Periodos.

**Entrada:** sin body; path { "id": "42" }.

**Salida:**

```
{"rutaId": "42", "estado": "PENDIENTE", "ordenesTrabajo": []}
```

### 5.8.7. Caso: Actualizar ruta

**Descripción:** modifica los datos de una ruta existente.

**HTTP y ruta:** PATCH /api/v1/routes/:id.

**Permiso:** routes:update.

**Cadena:** RoutesController.update() RoutesService.update() UpdateRouteUseCase.execute() repositorio.

**Errores relevantes:** 400; 401; 403; 404.

**Efectos:** actualización de Rutas; DTO puro.

**Prisma:** Rutas.

**Entrada:** path { "id": "42" }; body { "nombre": "Ruta actualizada" }.

**Salida:** { "rutaId": "42", "nombre": "Ruta actualizada", "estado": "PENDIENTE" }.

### 5.8.8. Caso: Reasignar ruta

**Descripción:** asigna o desasigna la ruta a un operario.

**HTTP y ruta:** PATCH /api/v1/routes/:id/reassign.

**Permiso:** routes:update.

**Cadena:** RoutesController.reassign() ReassignRouteUseCase.execute() repositorio.

**Errores relevantes:** 400; 401; 403; 404; ruta ya asignada.

**Efectos:** cambia Rutas.operarioId.

**Prisma:** Rutas, Usuarios/Operarios.

**Entrada:** path { "id": "42" }; body { "operarioId": "7" } o null.  
**Salida:** { "rutaId": "42", "operarioId": 7, "estado": "PENDIENTE" }.

#### 5.8.9. Caso: Eliminar ruta

**Descripción:** elimina lógicamente la ruta.

**HTTP y ruta:** DELETE /api/v1/routes/:id.

**Permiso:** routes:delete.

**Cadena:** RoutesController.delete() RoutesService.delete() DeleteRouteUseCase.execute()  
repositorio.

**Errores relevantes:** 400; 401; 403; 404.

**Efectos:** soft delete y desasignación según el caso.

**Prisma:** Rutas, OrdenTrabajo, Lecturas.

**Entrada:** sin body; path { "id": "42" }.

**Salida:** { "rutaId": "42", "estado": "CANCELADA" }.

#### 5.8.10. Caso: Exportar hoja de campo

**Descripción:** genera la hoja oficial de órdenes y lecturas.

**HTTP y ruta:** GET /api/v1/routes/:id/pdf.

**Permiso:** routes:read.

**Cadena:** RoutesController.exportPdf() RoutesService.exportPdf() ExportFieldSheetPdfUseCase/g

**Errores relevantes:** 400; 401; 403; 404.

**Efectos:** lectura y generación puras.

**Prisma:** Rutas, OrdenTrabajo, Lecturas.

**Entrada:** sin body; path { "id": "42" }.

**Salida:** no es JSON: application/pdf, Content-Disposition: inline, Content-Length y  
bytes PDF.

#### 5.8.11. Caso: Consultar órdenes de una ruta

**Descripción:** lista órdenes asociadas, filtradas y paginadas.

**HTTP y ruta:** GET /api/v1/routes/:id/work-orders.

**Permiso:** routes:read.

**Cadena:** RoutesController.findOrdenesByRuta() OrdenesTrabajoService.findByRuta()  
FindOrdenesByRutaUseCase.execute().

**Errores relevantes:** 400; 401; 403; 404.

**Efectos:** ninguno.

**Prisma:** OrdenTrabajo, Rutas, Lecturas.

**Entrada:** sin body; path { "id": "42" }; query estado, page, limit.

**Salida:** { "data": [{ "ordenTrabajoId": "9", "estado": "PENDIENTE" }], "meta":  
{}, "kpis": {} }.

#### 5.8.12. Caso: Actualizar estado de orden

**Descripción:** cambia estado, observación y fecha de finalización de una orden.

**HTTP y ruta:** PATCH /api/v1/work-orders/:id/state.

**Permiso:** routes:update.

**Cadena:** OrdenesTrabajoController.updateEstado() OrdenesTrabajoService.updateEstado()  
UpdateOrdenEstadoUseCase.execute() repositorio.

**Errores relevantes:** 400; 401; 403; 404; 409.

**Efectos:** persiste estado; completa o limpia completadoEn.

**Prisma:** OrdenTrabajo, EjecucionOrdenTrabajo.

**Entrada:** path { "id": "9" }; body { "estado": "COMPLETADA", "resultadoObservacion": "Trabajo realizado" }.

**Salida:** { "ordenTrabajoId": "9", "estado": "COMPLETADA" }.

#### 5.8.13. Caso: Vincular lectura a orden

**Descripción:** asocia una lectura y completa la orden.

**HTTP y ruta:** PATCH /api/v1/work-orders/:id/reading.

**Permiso:** routes:update.

**Cadena:** OrdenesTrabajoController.linkLectura() OrdenesTrabajoService.linkLectura() LinkLecturaUseCase.execute() repositorio.

**Errores relevantes:** 400; 401; 403; 404; 409.

**Efectos:** persiste vínculo y completadoEn.

**Prisma:** OrdenTrabajo, Lecturas, Rutas.

**Entrada:** path { "id": "9" }; body { "lecturaId": "101" }.

**Salida:** { "ordenTrabajoId": "9", "lecturaId": "101", "estado": "COMPLETADA" }.

#### 5.8.14. Caso: Actualizar operación del operador

**Descripción:** registra ejecución y evidencia opcional de una orden.

**HTTP y ruta:** PATCH /api/v1/operator/work-orders/:id.

**Permiso:** routes:update.

**Cadena:** OperatorController.updateOperatorWorkOrder() UpdateOperatorWorkOrderUseCase.execute() repositorio/RustFS.

**Errores relevantes:** 400; 401; 403; 404; 409.

**Efectos:** persiste ejecución y evidenciaFotoUrl; la subida y rollback de archivo son efectos reales.

**Prisma:** OrdenTrabajo, EjecucionOrdenTrabajo, NovedadOrdenTrabajo.

**Entrada:** path { "id": "9" }; multipart: campos DTO y archivo opcional foto; sin body JSON.

**Salida:** JSON OrderWorkResponseDto, por ejemplo { "ordenTrabajoId": "9", "estado": "COMPLETADA", "evidenciaFotoUrl": "ordenes/9/foto.jpg" }.

#### 5.8.15. Caso: Listar rutas del operador

**Descripción:** lista las rutas del período activo asignadas al operador autenticado.

**HTTP y ruta:** GET /api/v1/operator/routes.

**Permiso:** routes:read.

**Cadena:** OperatorController.getOperatorRoutes() GetOperatorRoutesUseCase.execute() repositorio.

**Errores relevantes:** 400; 401; 403; 404 sin período activo.

**Efectos:** ninguno; consulta y DTO puros.

**Prisma:** Rutas, OrdenTrabajo, Periodos.

**Entrada:** sin body; query opcional tipoRuta; operador desde JWT.

**Salida:** lista JSON de OperatorRouteResponseDto.

#### 5.8.16. Caso: Actualizar estado de ruta del operador

**Descripción:** cambia el estado de una ruta que pertenece al operador autenticado.

**HTTP y ruta:** PATCH /api/v1/operator/routes/:id/state.

**Permiso:** routes:update.

**Cadena:** OperatorController.updateRouteState() UpdateRouteStateUseCase.execute() repositorio.

**Errores relevantes:** 400; 401; 403 ruta ajena; 404; 409 transición inválida.

**Efectos:** actualiza Rutas y las marcas temporales derivadas del estado.

**Prisma:** Rutas, OrdenTrabajo, Periodos.

**Entrada:** path { "id": "42" }; body { "estado": "EN\_PROGRESO" }.

**Salida:** { "rutaId": "42", "estado": "EN\_PROGRESO" }.

## 5.9. Pendientes funcionales

Bloqueos confirmados como pendientes en el código revisado. Cuando se cierre cada uno, sacar de acá y mover a la sección correspondiente.

- **Bloqueo por novedad activa al transicionar la orden a COMPLETADA.** El helper `applyContractLifecycleTransition` solo valida estado origen del contrato, no valida `tieneNovedadActiva`. Falta decidir el criterio bloqueante (`afectaOrden`, tipo de novedad o resolución) e implementar la validación en `UpdateOrdenEstadoUseCase` o en el repositorio de órdenes.
- **Autorización exclusiva del rol Secretaría para la estimación con los últimos 3 meses.** Mismo punto pendiente en `05-lecturas.md`. Requiere endpoint nuevo o permiso específico (`estimates:approve` o equivalente). La política está acordada pero no está aplicada en código.
- **Conteo automático de órdenes para el cierre de la ruta (COMPLETADA vs PARCIAL).** Si TODAS las órdenes están COMPLETADA (incluidas completadas por estimación), la ruta pasa a COMPLETADA; si alguna está FALLIDA o CANCELADA, pasa a PARCIAL. Hoy `UpdateRouteUseCase` no implementa esa lógica; el estado final lo asigna manualmente quien cierre la ruta.
- **Endpoints HTTP para gestión manual de NovedadOrdenTrabajo.** Hoy la entidad solo tiene repositorio; la creación y actualización de novedades ocurre internamente desde otros flujos (captura de lecturas, reporte de defectos). Si el operador debe poder crear/actualizar/resolver novedades desde la UI, falta el controller + use cases correspondientes.
- **Auditoría de la decisión de secretaría al aprobar una ruta para EN\_PROGRESO** (usuario y fecha). El documento lo menciona en la sección de flujo acordado de la ruta pero no indica tabla ni columnas donde se persiste esa decisión.
- **Tipo de novedad INSPECCION sin efecto sobre ciclo de vida del contrato.** El helper `applyContractLifecycleTransition` solo reconoce los tipos `INSTALACION` y `RECONEXION`. Si una orden de inspección tiene contrato asociado, el efecto sobre el estado del contrato queda indefinido.

## 5.10. No documentado o pendiente de confirmar

- Lista exhaustiva de transiciones de operador.
- Endpoints legacy basados en `EstadoAsignacion`.

## 6. Lecturas y consumo

Captura, revisión, cálculo de consumo, anomalías y evidencia de lecturas.

**Ficha de dominio:** documenta consulta, captura por operador, revisión administrativa, cálculo de consumo, anomalías y evidencia vinculada a la orden. Consumo no es un dominio ni módulo HTTP independiente: se deriva de Lectura mediante sus mismos endpoints y casos de uso.

### 6.1. Separación entre orden y lectura

Para una orden de tipo LECTURA, el flujo confirmado separa la operación de campo de la validación del dato:

1. Ruta en EN\_PROGRESO → orden de lectura en EN\_PROGRESO.
2. El operador registra el dato → orden COMPLETADA.

3. La lectura queda en PENDIENTE/POR\_REVISION → APROBADA o RECHAZADA\_VERIFICACION.
4. Con medidor dañado, el flujo sigue la estimación autorizada (ver sección de estimación y [images/04-orden-lectura-flujo.png](#)).

La orden completada indica que terminó el trabajo de campo. La lectura APROBADA es la que puede alimentar la prefacturación; completar la orden no aprueba automáticamente el dato. ESTIMADA existe en el catálogo y su transición por estimación autorizada de secretaría es una política acordada pendiente de implementación (ver sección de estimación). Si existe una anomalía, CON\_NOVEDAD relaciona la lectura con `NovedadOrdenTrabajo`, pero no reemplaza el estado de la orden.

## 6.2. Alcance y entradas HTTP

- **Controladores:** `ReadingController`, `OperatorController`.
- **Servicio:** `ReadingService`.
- **Casos:** `FindAllReadingsUseCase`, `FindOneReadingUseCase`, `UpdateReadingUseCase`, `RemoveReadingUseCase`, `GetOperatorReadingsUseCase`, `GetOperatorReadingsWithAnomaliesUseCase`, `UpdateOperatorReadingUseCase`.
- El catálogo GET /estados se construye desde el enum y no tiene servicio/caso dedicado.

## 6.3. Comportamiento de negocio verificable

Las tablas relacionadas son `Lecturas`, `Contratos`, `Medidores`, `Periodos`, `Rutas`, `OrdenTrabajo` y `NovedadOrdenTrabajo`. El flujo confirmado es captura → revisión → aprobación/rechazo; el operador puede agregar anomalía y evidencia. La fórmula confirmada es `consumoCalculado = lecturaActual - lecturaAnterior`. Las lecturas APROBADA alimentan la prefacturación vía `generate-batch.use-case.ts:40`; lo que no está implementado es la promoción a PLANILLADA. No se encontró un handler propio de consumo.

| Método y ruta                           | Caso/servicio ejecutado                                                | Resultado                          |
|-----------------------------------------|------------------------------------------------------------------------|------------------------------------|
| GET /api/v1/readings                    | <code>findAll()</code><br><code>FindAllReadingsUseCase</code>          | Página.                            |
| GET /api/v1/readings/estados            | <code>getEstados()</code><br><code>buildStateCatalog()</code>          | Catálogo.                          |
| GET /api/v1/readings/:id                | <code>findOne()</code><br><code>FindOneReadingUseCase</code>           | Lectura.                           |
| PATCH /api/v1/readings/:id              | <code>update()</code><br><code>UpdateReadingUseCase</code>             | Lectura actualizada.               |
| DELETE /api/v1/readings/:id             | <code>delete()</code><br><code>RemoveReadingUseCase/repositorio</code> | Soft delete.                       |
| GET /api/v1/operator/readings           | <code>GetOperatorReadingsUseCase</code>                                | Lecturas propias.                  |
| GET /api/v1/operator/readings/anomalies | <code>GetOperatorReadingsWithAnomaliesUseCase</code>                   | Lecturas con anomalías pendientes. |
| PATCH /api/v1/operator/readings/:id     | <code>UpdateOperatorReadingUseCase</code>                              | Lectura/evidencia.                 |

## 6.4. Ejemplo JSON de respuesta

El contrato actual de lectura expone `estado`, `routeEstado` y `tieneAnomalia`. La orden relacionada se muestra como contexto propuesto porque `ResponseReadingDto` no expone actualmente un campo de orden anidado.

```
{
  "lecturaId": "101",
  "fecha": "2026-08-19",
  "lecturaAnterior": 500,
  "lecturaActual": 530,
  "consumoCalculado": 30,
  "estado": "PENDIENTE",
  "routeEstado": "EN_PROGRESO",
  "tieneAnomalia": false,
  "ordenRelacionada": {
    "ordenTrabajoId": "9",
    "estado": "COMPLETADA"
  }
}
```

**Campo propuesto:** ordenRelacionada permite representar la relación y el estado de la orden en una respuesta combinada, pero no debe interpretarse como un campo disponible del ResponseReadingDto actual.

| Campo                                                  | Tipo        | Regla/uso                                      |
|--------------------------------------------------------|-------------|------------------------------------------------|
| lecturaId                                              | string      | BigInt serializado.                            |
| fecha                                                  | string      | Fecha de lectura.                              |
| lecturaAnterior,<br>lecturaActual,<br>consumoCalculado | number      | Valores y diferencia de consumo.               |
| estado                                                 | enum        | Revisión y resultado.                          |
| evidenciaFotoUrl                                       | string/null | Evidencia de la orden vinculada cuando aplica. |

## 6.5. Estados

| Estado                 | Significado real                              | Transiciones confirmadas                                         |
|------------------------|-----------------------------------------------|------------------------------------------------------------------|
| PENDIENTE              | Dato capturado pendiente de revisión.         | POR_REVISION, APROBADA                                           |
| POR_REVISION           | Dato que requiere validación administrativa.  | APROBADA, RECHAZADA_VERIFICACION, PENDIENTE                      |
| APROBADA               | Dato validado y elegible para prefacturación. | No se confirmó una transición saliente en esta máquina.          |
| RECHAZADA_VERIFICACION | Dato rechazado durante la verificación.       | PENDIENTE, POR_REVISION                                          |
| ESTIMADA               | Valor estimado para la lectura.               | Estado existente sin transición confirmada en la máquina actual. |
| PLANILLADA             | Lectura incorporada a una planilla.           | Estado existente sin transición confirmada en la máquina actual. |
| CON_NOVEDAD            | Lectura relacionada con una anomalía/novedad. | Estado existente sin transición confirmada en la máquina actual. |

Estas son transiciones de Lectura, no de Ruta ni de OrdenTrabajo.

### 6.5.1. Máquina de estados de la Lectura

La máquina está implementada como una tabla de transiciones explícita en `backend/src/metering/readings/don`

| Desde                   | Hacia permitidas                             |
|-------------------------|----------------------------------------------|
| PENDIENTE               | POR\_REVISION, APROBADA                      |
| POR\_REVISION           | APROBADA, RECHAZADA\_VERIFICACION, PENDIENTE |
| RECHAZADA\_VERIFICACION | PENDIENTE, POR\_REVISION                     |

La función `canTransitionReadingState(from, to)` valida la transición; `UpdateReadingUseCase.execute()` (líneas 62–79) la aplica con control de concurrencia CAS. Los estados **APROBADA** y **RECHAZADA\\_VERIFICACION** son **terminales** (sin transiciones salientes en la tabla). Los estados **ESTIMADA**, **PLANILLADA** y **CON\_NOVEDAD** están en el enum pero **no tienen transiciones definidas** en la tabla — **CON\_NOVEDAD** se asigna desde `prisma-operator.repository.ts:602, 983` cuando el operador reporta una novedad.

### 6.6. Responsabilidades por estado

| Máquina                                 | Responsabilidad                                        |
|-----------------------------------------|--------------------------------------------------------|
| <code>Rutas.estado</code>               | Representa el lote y la operación general.             |
| <code>OrdenesTrabajo.estado</code>      | Representa la ejecución de la tarea de campo.          |
| <code>Lecturas.estado</code>            | Representa la calidad y validación del dato capturado. |
| <code>NovedadOrdenTrabajo.estado</code> | Representa el análisis y resolución de la anomalía.    |

`NovedadOrdenTrabajo` tiene su propia máquina: **OPEN**, **IN\_PROGRESS**, **RESOLVED**, **CANCELLED**.

### 6.7. Efectos y transacciones

La actualización administrativa persiste `Lecturas` y protege el cambio con control de concurrencia. La operación del operador puede persistir lectura, novedad y evidencia en `RustFS`; si falla, elimina la foto recién subida. Consultas, resta de consumo y DTO son transformaciones puras. **DELETE** usa `soft delete`.

| Tabla Prisma                                                            | Uso                                          |
|-------------------------------------------------------------------------|----------------------------------------------|
| <code>Lecturas</code>                                                   | Valores, estado y <code>soft delete</code> . |
| <code>Contratos</code> , <code>Medidores</code> , <code>Periodos</code> | Contexto de lectura.                         |
| <code>OrdenTrabajo</code> , <code>NovedadOrdenTrabajo</code>            | Operación y anomalías.                       |

### 6.8. Cálculo de consumo

El consumo es un cálculo derivado de `Lectura`, no una entidad, estado, controlador ni módulo HTTP independiente:

```
consumoCalculado = lecturaActual - lecturaAnterior
```

La resta se evalúa al actualizar o proyectar la lectura. El resultado se persiste tal cual; un valor negativo **no** activa ninguna lógica automática (no existe el término SALTO\_NEGATIVO en el código). No introduce un estado nuevo en la máquina de Lectura. La resta y la proyección al DTO son puras: la persistencia corresponde a la actualización de Lecturas y a sus efectos explícitos sobre anomalías y evidencia.

**Promoción a prefactura vs a PLANILLADA:** una lectura en estado APROBADA ya alimenta la prefacturación hoy (vía `generate-batch.use-case.ts:40`, que filtra lecturas APROBADA para incluirlas en el lote). Lo que **no** ocurre es una transición de la lectura a PLANILLADA: tras la prefacturación, la lectura sigue en APROBADA. El estado PLANILLADA figura en el enum y en el catálogo de iconos pero no se asigna desde ningún use case.

## 6.9. Estimación autorizada por secretaría

**Política acordada, pendiente de implementación.** Cubre la orden de LECTURA con medidor dañado.

- Sólo secretaría autoriza la estimación, calculada con los últimos 3 meses.
- Si autoriza: la lectura pasa a ESTIMADA y la orden a COMPLETADA; la novedad de medidor dañado sigue en OPEN para un reemplazo posterior.
- Si no autoriza o no se puede estimar: la orden sigue bloqueada en EN\_PROGRESO con `tieneNovedadActiva=true`.
- ESTIMADA no es APROBADA: sólo APROBADA puede alimentar la prefacturación. La orden COMPLETADA no implica lectura APROBADA.
- CON\_NOVEDAD vincula la lectura con la novedad; POR\_REVISION sigue siendo la validación administrativa del dato normal.

## 6.10. Diagrama de flujo acordado

Resumen textual: máquina de Lectura confirmada en código, más la política de estimación.

- PENDIENTE → POR\_REVISION (revisión) o APROBADA (aprobación directa permitida).
- POR\_REVISION → APROBADA (aprobación), RECHAZADA\_VERIFICACION (rechazo) o PENDIENTE (devolución).
- RECHAZADA\_VERIFICACION → PENDIENTE (nueva captura) o POR\_REVISION (nueva revisión).
- CON\_NOVEDAD: lectura con anomalía vinculada a NovedadOrdenTrabajo. La transición **hacia** CON\_NOVEDAD sí está implementada (`prisma-operator.repository.ts:602, 983`); lo que no existe es transición **desde** CON\_NOVEDAD (es terminal en la tabla actual).
- ESTIMADA: valor estimado; la transición por estimación autorizada de secretaría es política acordada pendiente de implementación.
- PLANILLADA: lectura incorporada a una planilla (sin transición confirmada en código).

## 6.11. Casos de uso

### 6.11.1. Caso: Listar lecturas

**Descripción:** devuelve lecturas filtradas y paginadas.

**HTTP y ruta:** GET `/api/v1/readings`.

**Permiso:** `lecturas:read`.

**Cadena:** `ReadingController.findAll()` `ReadingService.findAll()` `FindAllReadingsUseCase.execute()` `repository`.

**Errores relevantes:** 400; 401; 403.

**Efectos:** ninguno; lectura y DTO puros.

**Prisma:** `Lecturas`, `Contratos`, `Medidores`, `Periodos`.

**Entrada:** sin body; `query` `page`, `limit`, `contratoId`, `estado`, `search`.

## Flujo acordado de Orden, Lectura y Novedad — medidor dañado y estimación

Política acordada: la autorización de estimación por secretaría y el bloqueo por novedad están pendientes de implementación.

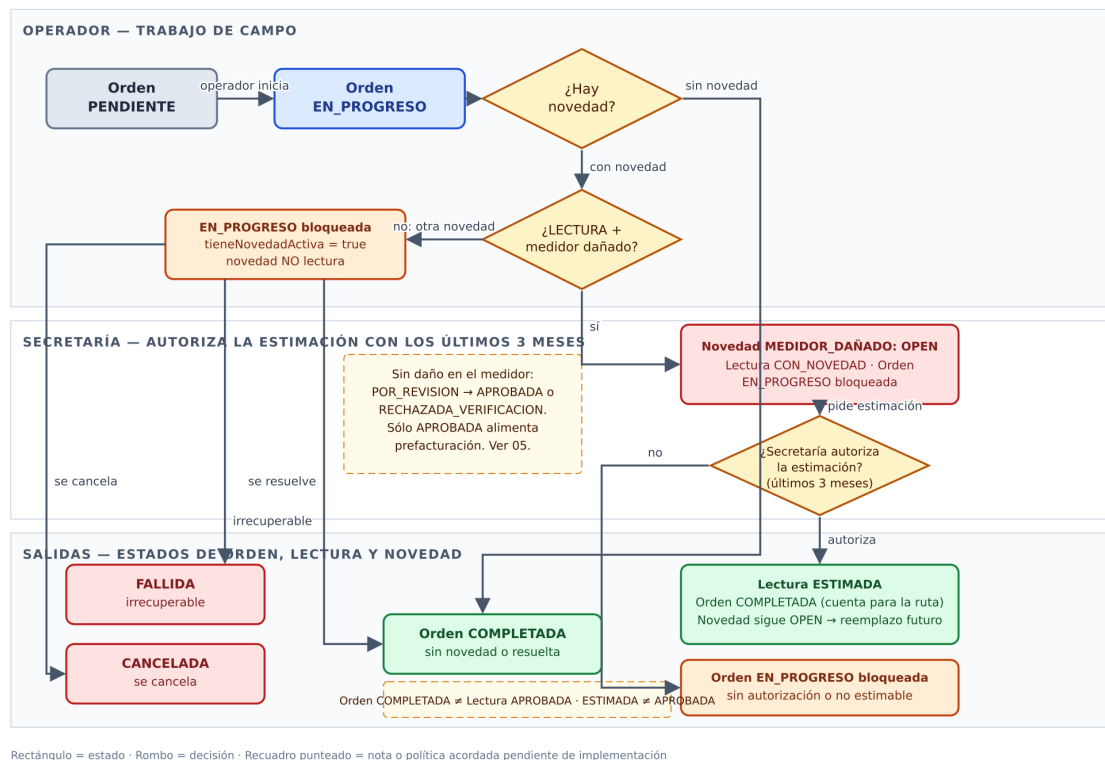


Figura 6: Flujo acordado de orden, lectura y novedad. Fuente editable: diagrams/orden-lectura-novedad.drawio.

**Salida:** { "data": [{ "lecturaId": "101", "estado": "PENDIENTE" }], "meta": {} }.

### 6.11.2. Caso: Consultar catálogo de estados

**Descripción:** entrega etiquetas e iconos de EstadoLectura.

**HTTP y ruta:** GET /api/v1/readings/estados.

**Permiso:** lecturas:read.

**Cadena:** ReadingController.getEstados() buildStateCatalog() (sin caso dedicado).

**Errores relevantes:** 401; 403.

**Efectos:** ninguno.

**Prisma:** ninguna tabla.

**Entrada:** sin body, path ni query.

**Salida:** [{ "codigo": "APROBADA", "descripcion": "Aprobada", "icono": "bi-check-circle" }].

### 6.11.3. Caso: Obtener lectura

**Descripción:** devuelve una lectura por ID.

**HTTP y ruta:** GET /api/v1/readings/:id.

**Permiso:** lecturas:read.

**Cadena:** ReadingController.findOne() ReadingService.findOne() FindOneReadingUseCase.execute repositorio.

**Errores relevantes:** 400; 401; 403; 404.

**Efectos:** ninguno; DTO puro.

**Prisma:** Lecturas, Medidores, Periodos, Contratos.

**Entrada:** sin body; path { "id": "101" }.

**Salida:** el JSON del ejemplo principal.

#### 6.11.4. Caso: Actualizar lectura administrativa

**Descripción:** actualiza campos no fotográficos y estado.

**HTTP y ruta:** PATCH /api/v1/readings/:id.

**Permiso:** lecturas:update.

**Cadena:** ReadingController.actualizarLectura() ReadingService.update()  
UpdateReadingUseCase.execute() repositorio.

**Errores relevantes:** 400 body/transición/CAS; 401; 403; 404; 409.

**Efectos:** persiste lectura; calcula consumoCalculado = lecturaActual - lecturaAnterior; la resta y el DTO son puros. El valor negativo se persiste tal cual (no hay lógica de SALTO\_NEGATIVO). Sólo APROBADA puede alimentar prefacturación; la promoción a PLANILLADA no está implementada.

**Prisma:** Lecturas, Medidores, OrdenTrabajo, NovedadOrdenTrabajo.

**Entrada:** path { "id": "101" }; body { "lecturaActual": 530, "estado": "POR\_REVISION" }.

**Salida:** { "lecturaId": "101", "lecturaActual": 530, "consumoCalculado": 30, "estado": "POR\_REVISION" }.

#### 6.11.5. Caso: Eliminar lectura

**Descripción:** aplica eliminación lógica.

**HTTP y ruta:** DELETE /api/v1/readings/:id.

**Permiso:** lecturas:delete.

**Cadena:** ReadingController.eliminarLectura() ReadingService.delete() RemoveReadingUseCase/re

**Errores relevantes:** 400; 401; 403; 404.

**Efectos:** establece deletedAt en Lecturas.

**Prisma:** Lecturas.

**Entrada:** sin body; path { "id": "101" }.

**Salida:** { "message": "Lectura eliminada" }.

#### 6.11.6. Caso: Listar lecturas del operador

**Descripción:** devuelve las lecturas del período activo asignadas al operador autenticado.

**HTTP y ruta:** GET /api/v1/operator/readings.

**Permiso:** lecturas:read.

**Cadena:** OperatorController.getOperatorReadings() GetOperatorReadingsUseCase.execute()  
repositorio y DTO.

**Errores relevantes:** 400; 401; 403; 404 sin período activo.

**Efectos:** ninguno; consulta y DTO puros.

**Prisma:** Lecturas, Rutas, OrdenTrabajo, Periodos.

**Entrada:** sin body ni query; operador desde JWT.

**Salida:** lista JSON de ResponseReadingDto.

#### 6.11.7. Caso: Listar anomalías pendientes del operador

**Descripción:** devuelve las lecturas del operador que tienen anomalías pendientes de atención.

**HTTP y ruta:** GET /api/v1/operator/readings/anomalies.

**Permiso:** lecturas:read.

**Cadena:** `OperatorController.getOperatorReadingsWithAnomalies()` `GetOperatorReadingsWithAnomalies()` repositorios y DTO.

**Errores relevantes:** 400; 401; 403; 404 sin período activo.

**Efectos:** ninguno; consulta y DTO puros.

**Prisma:** Lecturas, Rutas, OrdenTrabajo, NovedadOrdenTrabajo, Periodos.

**Entrada:** sin body ni query; operador desde JWT.

**Salida:** lista JSON, por ejemplo [ { "lecturaId": "101", "estado": "CON\_NOVEDAD" } ].

### 6.11.8. Caso: Actualizar lectura del operador

**Descripción:** captura valor, fecha, anomalía y foto de campo. La captura puede incluir la anomalía y la foto, y persiste la evidencia vinculada a la orden de trabajo.

**HTTP y ruta:** PATCH `/api/v1/operator/readings/:id`.

**Permiso:** lecturas:update.

**Cadena:** `OperatorController.updateOperatorReading()` `UpdateOperatorReadingUseCase.execute()` repositorios/RustFS.

**Errores relevantes:** 400; 401; 403 fuera de ruta; 404; 409.

**Efectos:** persiste lectura/novedad y guarda foto como `evidenciaFotoUrl` de la orden; rollback si falla. La captura no crea un módulo ni estado de Consumo.

**Prisma:** Lecturas, NovedadOrdenTrabajo, OrdenTrabajo.

**Entrada:** path { "id": "101" }; multipart campos DTO y archivo opcional foto; sin body JSON.

**Salida:** { "lecturaId": "101", "estado": "POR\_REVISION", "evidenciaFotoUrl": "ordenes/9/foto.jpg" }.

### 6.12. Pendientes funcionales

Bloqueos confirmados como pendientes en el código revisado. Cuando se cierre cada uno, sacar de acá y mover a la sección correspondiente.

- **Detección de consumo negativo (SALTO\_NEGATIVO).** El doc dice que se “identifica como SALTO\_NEGATIVO” pero no existe lógica de detección. El valor se persiste como número negativo sin acción. Falta decidir si se rechaza la lectura, se marca para revisión o se documenta el comportamiento tal como es.
- **Transición APROBADA → PLANILLADA.** No hay use case que cambie el estado a PLANILLADA. Si se requiere planillar, hay que agregar el cambio en el flujo de cierre del lote (`generate-batch.use-case.ts` o un nuevo job).
- **Transición CON\_NOVEDAD → PENDIENTE/POR\_REVISION.** No existe en `reading-state.ts`. Si la UI debe permitir reabrir la lectura tras resolver la novedad, falta la transición y el use case correspondiente.
- **Transición ESTIMADA → ... por estimación autorizada de secretaría.** Mismo punto pendiente en `02-contratos.md`, `04-rutas-y-ordenes.md` y ahora también aquí. Requiere endpoint nuevo o permiso específico (`estimates:approve` o equivalente).
- **Autorización exclusiva del rol Secretaría para la estimación últimos 3 meses.** No existe endpoint ni permiso que limite la estimación a secretaría.
- **Creación automática de anomalías según la superficie de captura.** Hoy la anomalía la crea el operador manualmente al reportar un defecto. No hay lógica que cree anomalía cuando, por ejemplo, el `lecturaActual` está fuera de un rango esperado.
- **Redondeo, consumo mínimo/máximo y otros límites.** `Prisma.Decimal` mantiene precisión arbitraria; no hay redondeo explícito ni límites configurados. Definir si aplica redondeo a 2 decimales, consumo mínimo configurable por categoría tarifaria o alertas por consumo atípico.

### 6.13. No documentado o pendiente de confirmar

- Alta independiente POST /readings (no existe en código; las lecturas se crean en flujos de reemplazo/instalación de medidores).
- Validación exacta del cuerpo de PATCH /api/v1/operator/readings/:id (multipart con archivo opcional foto). Los DTOs pueden haber cambiado desde la última revisión.

## 7. Medidores

Inventario, asignación, reemplazo y retiro operativo de medidores.

**Ficha de dominio:** documenta inventario, vínculo contractual, reemplazo, exportaciones, reporte de defectos y baja operativa del medidor.

### 7.1. Alcance y entradas HTTP

- **Controladores:** MeterController, OperatorController.
- **Servicio:** MeterService.
- **Casos:** CreateMeterUseCase, FindAllMetersUseCase, FindOneMeterUseCase, UpdateMeterUseCase, RemoveMeterUseCase, FindMeterHistoryUseCase, FindReplacementUseCase, ReplaceMeterUseCase, ExportMetersUseCase, ExportMetersPdfUseCase, ReportDefectUseCase, DecommissionMeterUseCase.

### 7.2. Comportamiento de negocio verificable

Las tablas relacionadas son Medidores, Contratos, HistorialMedidores, ReemplazoMedidor, Lecturas, OrdenTrabajo y NovedadOrdenTrabajo. El reemplazo es transaccional y conserva trazabilidad; los defectos/bajas cambian el estado del inventario. La transición del contrato asociado al completar una instalación (INSTALACION) o reconexión (RECONEXION) **sí** existe — está implementada vía applyContractLifecycleTransition (backend/src/operations/routes/infrastructure/repositories/p y documentada en 02-contratos.md, sección “Transiciones de estado del contrato”.

| Método y ruta                                | Caso/servicio ejecutado       | Resultado                |
|----------------------------------------------|-------------------------------|--------------------------|
| GET /api/v1/meters/status                    | MeterService.findAllStates()  | Catálogo.                |
| POST /api/v1/meters                          | create() CreateMeterUseCase   | Alta.                    |
| GET /api/v1/meters/:id                       | findAll()/findOne() consultas | Inventario/detalle.      |
| PATCH /api/v1/meters/:id                     | update() UpdateMeterUseCase   | Actualización.           |
| DELETE /api/v1/meters/:id                    | remove() RemoveMeterUseCase   | Soft delete.             |
| GET /api/v1/meters/export/csv / export/pdf   | exportación CSV/PDF           | Archivo.                 |
| GET /api/v1/meters/:id/history               | FindMeterHistoryUseCase       | Historial.               |
| GET /api/v1/meters/replacements/:id          | FindReplacementUseCase        | Reemplazo.               |
| POST /api/v1/meters/replace                  | ReplaceMeterUseCase           | Reemplazo transaccional. |
| POST /api/v1/meters/replacements/:id/approve | aprobación del servicio       | Resolución aprobada.     |
| POST /api/v1/operator/:id/report-defect      | ReportDefectUseCase           | DANADO.                  |
| POST /api/v1/operator/:id/decommission       | DecommissionMeterUseCase      | BAJA.                    |

### 7.3. Ejemplo JSON

```
{
  "medidorId": "10",
  "serie": "MED-12345",
  "marca": "Elster",
  "modelo": "V200",
  "estado": "INSTALADO"
}
```

| Campo            | Tipo        | Regla/uso                              |
|------------------|-------------|----------------------------------------|
| medidorId        | string      | BigInt serializado.                    |
| serie            | string      | Identificador único del inventario.    |
| marca, modelo    | string      | Datos del equipo.                      |
| estado           | enum        | Ciclo operativo.                       |
| evidenciaFotoUrl | string/null | Evidencia de orden cuando corresponde. |

### 7.4. Estados

| Estado    | Significado                    |
|-----------|--------------------------------|
| BODEGA    | Disponible.                    |
| PENDIENTE | Reservado durante instalación. |
| INSTALADO | En servicio.                   |
| DANADO    | Defecto reportado.             |
| BAJA      | Retirado del inventario.       |

#### 7.4.1. Máquina de estados del Medidor

A diferencia de Rutas o Lecturas, los medidores **no tienen una tabla de transiciones formal** en un archivo único. Las transiciones válidas están codificadas en cada use case. Las reglas confirmadas son:

| Desde              | Hacia permitidas | Validación                                                                                                          |
|--------------------|------------------|---------------------------------------------------------------------------------------------------------------------|
| [*]                | BODEGA           | al crear<br>(CreateMeterUseCase.execute()<br>línea 23)                                                              |
| BODEGA o PENDIENTE | INSTALADO        | al instalar/reemplazar<br>(prisma-meter.repository.ts:replaceMeter)                                                 |
| INSTALADO          | DANADO           | solo desde INSTALADO —<br>ReportDefectUseCase:26-30<br>rechaza otros estados con<br>InvalidDomainOperationException |
| DANADO             | BAJA             | solo desde DANADO —<br>DecommissionMeterUseCase:23-30<br>rechaza otros estados                                      |
| BAJA               | (ninguna)        | <b>No hay reversión ni reactivación.</b> Confirmado por búsqueda exhaustiva.                                        |

| Desde     | Hacia permitidas | Validación                                                                 |
|-----------|------------------|----------------------------------------------------------------------------|
| INSTALADO | BAJA directo     | <b>No existe.</b> Hay que pasar por DANADO (vía reporte de defecto) antes. |

El doc muestra el flujo como lineal; la realidad es que INSTALADO → BAJA directo está prohibido por el código. El diagrama mermaid debajo conserva el flujo lineal por simplicidad pero la tabla refleja las restricciones reales.

#### 7.4.2. Estados internos de ReemplazoMedidor

La entidad ReemplazoMedidor tiene DOS enums de estado propios, distintos del estado del medidor:

`EstadoResolucionConsumo` (backend/src/shared/enums/index.ts):

| Valor     | Significado                                       |
|-----------|---------------------------------------------------|
| PENDIENTE | Reemplazo registrado, aún sin aplicar al consumo. |
| APLICADA  | Consumo aplicado según el tratamiento definido.   |
| ANULADA   | Reemplazo anulado; no afecta al consumo.          |

`EstadoAprobacionReemplazo` (backend/src/shared/enums/index.ts):

| Valor     | Significado                                                     |
|-----------|-----------------------------------------------------------------|
| PENDIENTE | Reemplazo con tratamiento excepcional esperando aprobación.     |
| APROBADA  | Tratamiento autorizado por un usuario distinto del solicitante. |
| RECHAZADA | Tratamiento rechazado.                                          |

El reemplazo **estándar** (`TratamientoSaliente.COBR0\ \_REAL + TratamientoEntrante.FACTURAR\ \_PERIOD`) se autoaprueba en el mismo flujo de `replace-meter.use-case.ts:150-152`. El **excepcional** queda PENDIENTE hasta que otro usuario lo apruebe vía `POST /api/v1/meters/replacements/:id/approve`.

#### 7.5. Efectos y transacciones

Crear/actualizar/eliminar escriben Medidores; eliminar usa soft delete. Reemplazar registra reemplazo, historial, lecturas y resolución económica en la transacción del caso. Reportar defecto y dar de baja cambian estado y pueden crear tarea/novedad. Exportaciones, consultas y DTO son puras; CSV/PDF no son respuestas JSON.

| Tabla Prisma                      | Uso                               |
|-----------------------------------|-----------------------------------|
| Medidores                         | Inventario, estado y soft delete. |
| Contratos, HistorialMedidores     | Asignación y trazabilidad.        |
| ReemplazoMedidor                  | Ciclo de reemplazo.               |
| Lecturas                          | Lectura final/inicial.            |
| OrdenTrabajo, NovedadOrdenTrabajo | Inspección y evidencia.           |

## 7.6. Gráfico de estados

```
stateDiagram-v2
    [*] --> BODEGA: crear
    BODEGA --> PENDIENTE: vinculación
    PENDIENTE --> INSTALADO: instalación
    INSTALADO --> DANADO: reportar defecto
    DANADO --> BAJA: dar de baja
    INSTALADO -.no permitida.-> BAJA
    No se encontró una reactivación desde BAJA.
```

## 7.7. Casos de uso

### 7.7.1. Caso: Crear medidor

**Descripción:** registra un equipo en inventario.

**HTTP y ruta:** POST /api/v1/meters.

**Permiso:** meters:create.

**Cadena:** MeterController.create() MeterService.create() CreateMeterUseCase.execute()  
repositorio.

**Errores relevantes:** 400; 401; 403; 409 serie duplicada.

**Efectos:** alta en Medidores.

**Prisma:** Medidores.

**Entrada:** { "serie": "MED-12345", "marca": "Elster", "modelo": "V200" }.

**Salida:** { "medidorId": "10", "serie": "MED-12345", "estado": "BODEGA" }.

### 7.7.2. Caso: Listar medidores

**Descripción:** consulta el inventario paginado y filtrable.

**HTTP y ruta:** GET /api/v1/meters.

**Permiso:** meters:read.

**Cadena:** MeterController.findAll() MeterService.findAll() FindAllMetersUseCase.execute()  
repositorio.

**Errores relevantes:** 400; 401; 403.

**Efectos:** ninguno; consulta y DTO puros.

**Prisma:** Medidores, relaciones contractuales.

**Entrada:** sin body; query page, limit, estado y búsqueda.

**Salida:** { "data": [{ "medidorId": "10", "estado": "INSTALADO" }], "meta": {}  
}.

### 7.7.3. Caso: Obtener medidor

**Descripción:** devuelve el detalle de un medidor por identificador.

**HTTP y ruta:** GET /api/v1/meters/:id.

**Permiso:** meters:read.

**Cadena:** MeterController.findOne() MeterService.findOne() FindOneMeterUseCase.execute()  
repositorio.

**Errores relevantes:** 400; 401; 403; 404.

**Efectos:** ninguno; DTO puro.

**Prisma:** Medidores, relaciones contractuales.

**Entrada:** sin body; path { "id": "10" }.

**Salida:** objeto MeterResponseDto, como el ejemplo principal.

#### 7.7.4. Caso: Actualizar medidor

**Descripción:** modifica datos permitidos del inventario.

**HTTP y ruta:** PATCH /api/v1/meters/:id.

**Permiso:** meters:update.

**Cadena:** MeterController.update() MeterService.update() UpdateMeterUseCase.execute()  
repositorio.

**Errores relevantes:** 400; 401; 403; 404; 409 serie duplicada.

**Efectos:** actualiza Medidores; DTO puro.

**Prisma:** Medidores.

**Entrada:** path { "id": "10" }; body { "marca": "Itron" }.

**Salida:** { "medidorId": "10", "marca": "Itron", "estado": "BODEGA" }.

#### 7.7.5. Caso: Eliminar medidor

**Descripción:** marca el medidor como eliminado lógicamente.

**HTTP y ruta:** DELETE /api/v1/meters/:id.

**Permiso:** meters:delete.

**Cadena:** MeterController.delete() MeterService.remove() RemoveMeterUseCase.execute()  
repositorio.

**Errores relevantes:** 400; 401; 403; 404.

**Efectos:** establece deletedAt.

**Prisma:** Medidores.

**Entrada:** sin body; path { "id": "10" }.

**Salida:** { "message": "Medidor eliminado correctamente" }.

#### 7.7.6. Caso: Reemplazar medidor

**Descripción:** cambia el equipo de un contrato y conserva trazabilidad de lecturas y resolución económica.

**HTTP y ruta:** POST /api/v1/meters/replace.

**Permiso:** meters:update.

**Cadena:** MeterController.replace() MeterService.replaceMeter() ReplaceMeterUseCase.execute()  
transacción/repositorios.

**Errores relevantes:** 400; 401; 403; 404; lectura inconsistente.

**Efectos:** reemplazo, historial y lecturas en transacción.

**Prisma:** ReemplazoMedidor, Medidores, HistorialMedidores, Lecturas, Contratos.

**Entrada:** body ReplaceMeterDto con contrato, medidores y lecturas final/inicial.

**Salida:** { "reemplazoMedidorId": "4", "medidorAnteriorId": "10", "medidorNuevoId":  
"11" }.

#### 7.7.7. Caso: Obtener reemplazo de medidor

**Descripción:** consulta el detalle de un reemplazo registrado.

**HTTP y ruta:** GET /api/v1/meters/replacements/:id.

**Permiso:** meters:read.

**Cadena:** MeterController.findReplacement() MeterService.findReplacement() repo-  
sitorio.

**Errores relevantes:** 400; 401; 403; 404.

**Efectos:** ninguno; consulta y DTO puros.

**Prisma:** ReemplazoMedidor, Medidores, Contratos.

**Entrada:** sin body; path { "id": "4" }.

**Salida:** { "reemplazoMedidorId": "4", "medidorAnteriorId": "10", "medidorNuevoId": "11" }.

#### 7.7.8. Caso: Aprobar reemplazo de medidor

**Descripción:** aprueba el tratamiento económico o técnico de un reemplazo pendiente.

**HTTP y ruta:** POST /api/v1/meters/replacements/:id/approve.

**Permiso:** meter-replacements:approve.

**Cadena:** MeterController.approveReplacement() MeterService.approveReplacement()  
caso/repositorio de aprobación.

**Errores relevantes:** 400; 401; 403; 404; conflicto de resolución.

**Efectos:** en una transacción, valida identidad del aprobador (debe ser distinto del solicitante), confirma idempotencia si ya estaba aprobada, valida que esté en PENDIENTE, promueve las lecturas vinculadas a APROBADA con fechaValidacion, y persiste estadoAprobacion: APROBADA, autorizadoPorUsuarioId, autorizadoEn. Si la combinación de tratamientos es “estándar” (no requiereAprobacion), el reemplazo se autoaprueba al crearse; no es necesario llamar a este endpoint en ese caso.

**Prisma:** ReemplazoMedidor, Lecturas (vinculadas), Medidores, Contratos.

**Entrada:** sin body; path { "id": "4" }.

**Salida:** { "reemplazoMedidorId": "4", "estado": "APROBADO" }.

#### 7.7.9. Caso: Exportar inventario CSV

**Descripción:** descarga el inventario filtrado en formato CSV.

**HTTP y ruta:** GET /api/v1/meters/export/csv.

**Permiso:** meters:read.

**Cadena:** MeterController.exportCsv() MeterService.exportCsv() ExportMetersUseCase.execute()

**Errores relevantes:** 400; 401; 403; error de generación.

**Efectos:** sólo lectura y generación.

**Prisma:** Medidores y relaciones seleccionadas.

**Entrada:** sin body; query ExportMeterDto.

**Salida:** no es JSON: Content-Type: text/csv; charset=utf-8, Content-Disposition: attachment y contenido CSV.

#### 7.7.10. Caso: Exportar inventario PDF

**Descripción:** genera un reporte PDF del inventario filtrado.

**HTTP y ruta:** GET /api/v1/meters/export/pdf.

**Permiso:** meters:read.

**Cadena:** MeterController.exportPdf() MeterService.exportPdf() ExportMetersPdfUseCase.execute()

**Errores relevantes:** 400; 401; 403; error de generación.

**Efectos:** sólo lectura y generación.

**Prisma:** Medidores y relaciones seleccionadas.

**Entrada:** sin body; query ExportMeterDto.

**Salida:** no es JSON: application/pdf, Content-Disposition: attachment, Content-Length y bytes PDF.

#### 7.7.11. Caso: Reportar defecto de medidor

**Descripción:** registra que un medidor presenta un defecto operativo.

**HTTP y ruta:** POST /api/v1/operator/:id/report-defect.

**Permiso:** meters:update.

**Cadena:** `OperatorController.reportDefect() ReportDefectUseCase.execute() repositorio.`

**Errores relevantes:** 400 estado/ID; 401; 403; 404; 409.

**Efectos:** actualiza el estado a DANADO y puede crear novedad/evidencia.

**Prisma:** Medidores, NovedadOrdenTrabajo, OrdenTrabajo.

**Entrada:** path { "id": "10" }; body según el DTO de defecto.

**Salida:** `MeterResponseDto`, por ejemplo { "medidorId": "10", "estado": "DANADO" }.

#### 7.7.12. Caso: Dar de baja un medidor

**Descripción:** retira un medidor del inventario operativo.

**HTTP y ruta:** POST /api/v1/operator/:id/decommission.

**Permiso:** meters:delete.

**Cadena:** `OperatorController.decommissionMeter() DecommissionMeterUseCase.execute() repositorio.`

**Errores relevantes:** 400 estado/ID; 401; 403; 404; 409.

**Efectos:** actualiza el estado a BAJA y registra el motivo/historial.

**Prisma:** Medidores, historial y auditoría relacionada.

**Entrada:** path { "id": "10" }; body { "motivoBaja": "Replacement" }.

**Salida:** `MeterResponseDto`, por ejemplo { "medidorId": "10", "estado": "BAJA" }.

### 7.8. Pendientes funcionales

Bloqueos confirmados como pendientes en el código revisado. Cuando se cierre cada uno, sacar de acá y mover a la sección correspondiente.

- **Reversión/reactivación desde BAJA.** Sigue sin haber camino de vuelta. Confirmado por búsqueda exhaustiva; no hay caso de uso ni endpoint que cambie BAJA a otro estado. Si se requiere reactivación, hay que agregar el flujo correspondiente.
- **Forma exacta de los DTOs report-defect y decommission.** El doc muestra solo un ejemplo parcial del body. Confirmar contra el código actual los campos obligatorios y opcionales.
- **Reglas de timeout y notificación para reemplazos pendientes de aprobación.** No se detectó mecanismo de timeout ni notificación automática para reemplazos en EstadoAprobacionReemplazo: PENDIENTE. Si el aprobador nunca actúa, el reemplazo queda esperando indefinidamente.
- **Estados completos de EstadoAsignacion** (NO\\_ASIGNADA | ASIGNADA | TOMADA | COMPLETADA) y su rol actual. Marcado como legacy en 04-rutas-y-ordenes.md. Sigue sin clarificación sobre si aplica al flujo de medidores.

### 7.9. No documentado o pendiente de confirmar

- Forma exacta de los DTOs replace y find-replacement. Confirmar contra el código actual.

## 8. Categorías de tarifa

Clasificación tarifaria, vigencias y rubros aplicables al contrato.

**Ficha de dominio:** documenta altas, consultas, versionado, desactivación y relaciones con rubros y contratos de las categorías tarifarias.

## 8.1. Alcance y entradas HTTP

- **Controlador:** CategoriaTarifaController.
- **Servicio:** CategoriaTarifaService.
- **Casos:** CreateTariffCategoryUseCase, FindAllTariffCategoriesUseCase, FindOneTariffCategoryUseCase, UpdateTariffCategoryUseCase, RemoveTariffCategoryUseCase.

## 8.2. Comportamiento de negocio verificable

Las tablas relacionadas son CategoriaTarifa, Rubros y Contratos. En instalación, la selección vigente del rubro es por categoría y codigoSistemaRubro=INSTALACION. No se encontró fórmula completa de consumo mensual ni handler que propague automáticamente una nueva versión a contratos existentes.

| Método y ruta                           | Caso/servicio ejecutado                                      | Resultado         |
|-----------------------------------------|--------------------------------------------------------------|-------------------|
| POST<br>/api/v1/tariff-categories       | createCategoria()<br>CreateTariffCategoryUseCase.execute()   | Categoría creada. |
| GET<br>/api/v1/tariff-categories        | getCategorias()<br>FindAllTariffCategoriesUseCase.execute()  | Página activa.    |
| GET<br>/api/v1/tariff-categories/:id    | findOneCategoria()<br>FindOneTariffCategoryUseCase.execute() | Detalle.          |
| PATCH<br>/api/v1/tariff-categories/:id  | updateCategoria()<br>UpdateTariffCategoryUseCase.execute()   | Nueva versión.    |
| DELETE<br>/api/v1/tariff-categories/:id | deleteCategoria()<br>RemoveTariffCategoryUseCase.execute()   | Soft delete.      |

## 8.3. Ejemplo JSON

```
{
  "categoriaTarifaId": 1,
  "nombre": "Residencial",
  "descripcion": "Tarifa básica",
  "consumoMinimoMensual": 10,
  "fechaVigenciaDesde": "2026-01-01",
  "fechaVigenciaHasta": null,
  "activo": true
}
```

| Campo                                     | Tipo        | Regla/uso                    |
|-------------------------------------------|-------------|------------------------------|
| categoriaTarifaId                         | number      | Identificador entero.        |
| nombre, descripcion                       | string      | Clasificación y explicación. |
| consumoMinimoMensual                      | number      | Mínimo configurado.          |
| fechaVigenciaDesde,<br>fechaVigenciaHasta | string/null | Intervalo de vigencia.       |
| activo                                    | boolean     | Disponibilidad lógica.       |

## 8.4. Estados

No existe enum de estado. La disponibilidad se expresa con activo y debe interpretarse junto con las fechas de vigencia. deletedAt es interno.

## 8.5. Efectos y transacciones

Crear escribe la categoría y puede generar rubros por defecto. Actualizar crea una nueva versión tarifaria, con fechas y rubros según el caso. Eliminar aplica soft delete/desactivación. Lecturas, filtros y `TariffCategoryResponseDto.fromEntity` son transformaciones puras.

| Tabla Prisma    | Uso                                  |
|-----------------|--------------------------------------|
| CategoriaTarifa | Categorías, versiones y soft delete. |
| Rubros          | Cargos relacionados.                 |
| Contratos       | Referencias contractuales.           |

## 8.6. Estados y transiciones no encontrados

No existe una máquina de estados confirmable ni un enum de estado. El código confirma los campos `\texttt{activo}`, las fechas de vigencia y `\texttt{deletedAt}`; también confirma que `\texttt{PATCH}` crea una nueva versión y que `\texttt{DELETE}` aplica desactivación/soft delete. No se dibujan flechas porque no existe una transición explícita verificable.

## 8.7. Casos de uso

### 8.7.1. Caso: Crear categoría de tarifa

**Descripción:** registra una categoría y sus rubros por defecto cuando corresponde.

**HTTP y ruta:** POST `/api/v1/tariff-categories`.

**Permiso:** `tarifas:create`.

**Cadena:** `CategoriaTarifaController.create() CategoriaTarifaService.createCategoria() CreateTariffCategoryUseCase.execute()` repositorio.

**Errores relevantes:** 400; 401; 403; conflicto de vigencia.

**Efectos:** alta en `CategoriaTarifa` + **creación automática de 2 Rubros default:** `codigoSri='001'` (“Cargo fijo”, `tipoRubro=FIJO`) y `codigoSri='002'` (“Consumo de agua potable”, `tipoRubro=VARIABLE`), ambos con `esAutomatico=true`. Si el DTO no trae `tarifaImpuestoId`, se toma la primera tarifa de impuesto activa.

**Prisma:** `CategoriaTarifa, Rubros`.

**Entrada:** `{ "nombre": "Residencial", "descripcion": "Tarifa básica", "consumoMinimoMensual": 10, "fechaVigenciaDesde": "2026-01-01" }`.

**Salida:** JSON del ejemplo principal.

### 8.7.2. Caso: Listar categorías

**Descripción:** devuelve categorías activas con filtros y paginación.

**HTTP y ruta:** GET `/api/v1/tariff-categories`.

**Permiso:** `tarifas:read`.

**Cadena:** `CategoriaTarifaController.findAll() CategoriaTarifaService.getCategorias() FindAllTariffCategoriesUseCase.execute()` repositorio.

**Errores relevantes:** 400; 401; 403.

**Efectos:** ninguno; consulta y DTO puros.

**Prisma:** `CategoriaTarifa`.

**Entrada:** sin body; query `page, limit, nombre, search`.

**Salida:** `{ "data": [{ "categoriaTarifaId": 1, "nombre": "Residencial", "activo": true }], "meta": {} }`.

### 8.7.3. Caso: Obtener categoría

**Descripción:** consulta una categoría por ID.

**HTTP y ruta:** GET /api/v1/tariff-categories/:id.

**Permiso:** tarifas:read.

**Cadena:** CategoriaTarifaController.findOne() CategoriaTarifaService.findOneCategoria()  
FindOneTariffCategoryUseCase.execute() repositorio.

**Errores relevantes:** 400; 401; 403; 404.

**Efectos:** ninguno; DTO puro.

**Prisma:** CategoriaTarifa, Rubros, Contratos si se incluyen.

**Entrada:** sin body; path { "id": 1 }.

**Salida:** JSON del ejemplo principal.

### 8.7.4. Caso: Actualizar categoría

**Descripción:** crea una nueva versión en vez de mutar silenciosamente la vigencia anterior.

**HTTP y ruta:** PATCH /api/v1/tariff-categories/:id.

**Permiso:** tarifas:update.

**Cadena:** CategoriaTarifaController.update() CategoriaTarifaService.updateCategoria()  
UpdateTariffCategoryUseCase.execute() repositorio.

**Errores relevantes:** 400 datos/vigencia; 401; 403; 404; solapamiento.

**Efectos:** persiste nueva versión y fechas; DTO puro.

**Prisma:** CategoriaTarifa, Rubros, referencias contractuales.

**Entrada:** path { "id": 1 }; body { "consumoMinimoMensual": 12, "fechaVigenciaDesde":  
"2026-10-01" }.

**Salida:** { "categoriaTarifaId": 2, "nombre": "Residencial", "consumoMinimoMensual":  
12, "activo": true }.

### 8.7.5. Caso: Eliminar categoría

**Descripción:** desactiva y elimina lógicamente la categoría.

**HTTP y ruta:** DELETE /api/v1/tariff-categories/:id.

**Permiso:** tarifas:delete.

**Cadena:** CategoriaTarifaController.remove() CategoriaTarifaService.deleteCategoria()  
RemoveTariffCategoryUseCase.execute() repositorio.

**Errores relevantes:** 400; 401; 403; 404; uso incompatible.

**Efectos:** triple efecto simultáneo: activo=false, fechaVigenciaHasta=now, deletedAt=now  
(no solo soft delete: también cierra la vigencia).

**Prisma:** CategoriaTarifa.

**Entrada:** sin body; path { "id": 1 }.

**Salida:** { "categoriaTarifaId": 1, "activo": false, "fechaVigenciaHasta":  
"<fecha actual al momento de la baja (now())>" }.

## 8.8. Pendientes funcionales

Bloqueos confirmados como pendientes en el código revisado. Cuando se cierre cada uno, sacar de acá y mover a la sección correspondiente.

- **Propagación automática de cambios a contratos existentes.** createNewVersion no toca la tabla Contratos. La nueva versión queda disponible para futuros contratos, pero los vigentes siguen apuntando a la versión anterior. No hay mecanismo de migración.

- **Reglas de solapamiento entre versiones.** Aunque “solapamiento” figuraba en los errores relevantes del caso Actualizar, el código **no lo valida**. Si se quiere esa regla, hay que decidir el criterio (¿no se permite que dos versiones activas tengan fechas superpuestas?) y agregar la validación.
- **Duración mínima o reglas de vigencia.** No hay reglas explícitas sobre `fechaVigenciaDesde`  $\geq$  hoy, sobre solapamiento con otras versiones, ni sobre la duración mínima de una versión.
- **Estados legacy distintos de activo, deletedAt y fechas.** Confirmar contra seeds y migraciones si existe alguna columna o enum legacy.

## 8.9. No documentado o pendiente de confirmar

- Forma exacta del DTO `update-categoria-tarifa` y de los campos opcionales que se propagan a la nueva versión. Confirmar contra el código actual.

## Referencias